

**Institut Universitaire de Technologie,
Aix-Marseille Université**

**RAPPORT DE STAGE de fin de deuxième année
Bachelor Universitaire de Technologie
Spécialité Réseaux et Télécommunications
parcours cybersécurité**

**Renforcement de la Sécurité Informatique et
Optimisation des Services IT**

Victor HACQUE--RANQUE

CAMPUS PROVENCE VERTE

Responsable entreprise : Ioane Ducresson

Responsable académique : Eric Würbel

2025

Table des matières

1 Introduction	5
2 Présentation de la structure d'accueil dans son secteur d'activité	6
2.1 Présentation du Campus Provence Verte	6
2.2 Les formations et les projets	6
2.3 Activités et organisation du service informatique	7
3 Présentation des missions réalisées	8
3.1 Sécurisation des serveurs	8
3.1.1 Contexte	8
3.1.2 Plus de détail sur les outils et technologies utilisés	10
3.1.3 Tâches effectués	13
3.1.4 Problèmes rencontrés et leur solution	23
3.2 Gestion centralisée des postes et des utilisateurs	27
3.2.1 Contexte	27
3.2.2 Plus de détail sur les outils et technologies utilisés	28
3.2.3 Tâches effectués	28
3.2.4 Problèmes rencontrés et leur solution	30
4 Conclusion	33
5 Remerciements	35
6 Glossaire	37
7 Sitographie	39

1 Introduction

Accueilli pendant dix semaines au sein du service informatique du Campus Provence Verte, j'ai pu évoluer dans un cadre technique exigeant, où coexistent établissements secondaires et supérieurs. Situé à Saint-Maximin-la-Sainte-Baume, ce campus gère une infrastructure numérique dense, nécessitant une supervision constante et une sécurisation poussée. Cette immersion s'est inscrite dans le prolongement de ma formation en deuxième année de Bachelor Universitaire de Technologie Réseaux & Télécommunications à l'IUT d'Aix-Marseille, me permettant de confronter mes acquis à la réalité d'un environnement de production.

Le stage visait à renforcer la sécurité de l'infrastructure informatique tout en améliorant l'efficacité de la supervision, des sauvegardes et de la gestion des postes et utilisateurs. J'ai ainsi eu l'opportunité de participer à un ensemble de projets concrets et complémentaires, en lien avec les thématiques étudiées au cours de ma formation. L'une de mes premières missions a consisté à réaliser un audit de sécurité sur les serveurs, à identifier les points faibles et à mettre en place des protections adaptées, notamment par l'installation de pare-feu, l'application régulière des mises à jour, et le durcissement des accès aux services critiques. J'ai ensuite été chargé de déployer une solution de supervision centralisée, en combinant Glances pour une interface simplifiée et Netdata pour plus de détails. Cette supervision a permis un suivi précis des performances du réseau et des serveurs, avec des alertes personnalisées en cas d'anomalie.

En parallèle, j'ai travaillé sur la mise en place de sauvegardes automatisées pour les données sensibles, en m'appuyant sur un NAS (Network Attached Storage, ou stockage sur réseau) Synology local. Des tests de restauration ont été réalisés afin de garantir l'intégrité et la fiabilité des procédures en cas de sinistre. J'ai également participé à la gestion centralisée des utilisateurs et des postes informatiques, en intégrant les machines à un domaine SambaEdu, en configurant les stratégies de groupe, et en automatisant certaines tâches récurrentes. D'autres missions secondaires m'ont été confiées, comme le support technique auprès des utilisateurs et la mise à jour des équipements logiciels du parc.

Ce rapport retrace l'ensemble de ces missions ainsi que les outils et moyens mis en œuvre pour les mener à bien. Il s'ouvre sur une présentation générale du Campus Provence Verte et de son service informatique, avant de détailler les actions menées pour sécuriser les serveurs et durcir leur configuration. Ensuite, il explore la mise en place de la supervision et des sauvegardes, avant d'aborder la gestion centralisée du parc informatique. Enfin, le rapport conclut sur les compétences acquises et les enseignements tirés de cette immersion professionnelle.

2 Présentation de la structure d'accueil dans son secteur d'activité

2.1 Présentation du Campus Provence Verte

Situé à Saint-Maximin-la-Sainte-Baume, au cœur du Var, en région Provence-Alpes-Côte d'Azur, le Campus Provence Verte est un établissement d'enseignement agricole privé sous contrat avec le Ministère de l'Agriculture et de la Souveraineté Alimentaire. Il s'inscrit dans le réseau de l'enseignement agricole français, qui regroupe plus de 800 établissements, dont près des trois quarts sont privés. Ce secteur forme chaque année plus de 150 000 élèves et 45 000 apprentis et constitue un levier stratégique pour la transition écologique et alimentaire. Il bénéficie à ce titre d'un important soutien public avec un budget dédié de plus de 2,2 milliards d'euros.

Le Campus Provence Verte a été fondé en 1953 par les Sœurs Dominicaines, un ordre religieux connu pour son engagement dans l'éducation et la transmission de valeurs humaines. À l'origine modeste, l'établissement a connu une évolution constante: il est reconnu officiellement en 1969 par le ministère, devient Centre d'Enseignement Rural Mixte en 1971, et obtient le statut de lycée agricole en 1981. Il est aujourd'hui membre du CNEAP, le Conseil National de l'Enseignement Agricole Privé, un réseau d'établissements qui mettent l'accent sur l'accompagnement individuel, la formation professionnelle, l'ouverture sociale et les valeurs humanistes.

L'établissement est géré par une association à but non lucratif. Cette association est composée de parents d'élèves, d'anciens élèves, de professionnels du monde agricole, de partenaires publics et privés, ainsi que de membres engagés dans la vie locale. Cette structure associative assure une gestion souple et adaptée, au plus près des besoins du territoire, avec une capacité d'évolution constante.

Le site offre un cadre de vie agréable et propice à l'apprentissage, avec des infrastructures modernes et fonctionnelles. L'internat permet d'accueillir jusqu'à 220 élèves. Il favorise la responsabilisation, la vie en collectivité et l'autonomie. Le campus dispose de nombreuses salles spécialisées, de laboratoires, d'un centre de documentation et d'information, d'espaces pédagogiques extérieurs, d'installations sportives et d'un restaurant scolaire. Certains produits servis à la cantine sont issus de l'exploitation pédagogique de l'établissement, ce qui implique directement les élèves dans un cycle vertueux et concret. Le cadre naturel et les outils pédagogiques de terrain sont pleinement intégrés à l'enseignement, dans une logique de pédagogie active.

Le fonctionnement du campus repose sur une hiérarchie claire et structurée. Il est dirigé par Monsieur Christian Brayer, directeur de l'établissement, qui assure la coordination globale. Il est suivi par des responsables de filières chargés des formations professionnelles et technologiques, un responsable de la vie scolaire, une cheffe des services administratifs, un responsable de l'exploitation pédagogique, et plusieurs référents ou coordonnateurs pédagogiques selon les filières. Cette organisation permet une gestion efficace et adaptée aux besoins spécifiques des différents pôles de l'établissement.

2.2 Les formations et les projets

Le Campus Provence Verte accueille aujourd'hui environ 730 élèves et étudiants, répartis sur différentes filières allant de la 4e de l'enseignement agricole aux formations post-bac. Il emploie environ 130 personnels, qu'il s'agisse de l'équipe éducative, du personnel administratif ou technique. L'offre de formation est très variée et adaptée à des profils d'élèves aux objectifs multiples. En collège, des classes de 4e et 3e permettent de raccrocher scolairement les élèves en difficulté en leur proposant une pédagogie plus concrète. En lycée professionnel, plusieurs baccalauréats professionnels sont proposés, notamment dans les domaines des services à la personne, de la production animale ou de la production viticole. En lycée technologique, on retrouve le bac STAV (Sciences et Technologies de l'Agronomie et du Vivant), qui propose une

formation polyvalente axée sur l'agronomie, la biologie, l'écologie et la technologie. Il existe également des formations post-bac, notamment plusieurs BTSA (Brevet de Technicien Supérieur Agricole) dans les domaines agricoles, environnementaux ou liés aux services, ainsi qu'un bac général. Cette diversité de parcours permet soit une insertion rapide dans le monde du travail, soit une poursuite d'études en licence, école spécialisée ou école d'ingénieur.

Dans cette logique d'évolution permanente et d'adaptation aux besoins de la société, le Campus Provence Verte s'est engagé depuis quelques années dans une double transition, à la fois écologique et numérique. Il met en œuvre diverses initiatives pour limiter son impact environnemental: tri des déchets, réduction des consommations, sensibilisation des élèves, valorisation des circuits courts. En parallèle, le numérique prend une place de plus en plus importante, aussi bien dans l'enseignement que dans l'administration. L'établissement a investi dans l'équipement des salles, la gestion informatique des emplois du temps, des évaluations, des bulletins, ainsi que dans des plateformes de communication avec les familles et les élèves. La transformation numérique ne concerne pas seulement la pédagogie: elle s'applique aussi aux infrastructures, à la sécurité informatique, aux systèmes de sauvegarde et à la gestion centralisée des postes.

2.3 Activités et organisation du service informatique

Le service informatique occupe une place stratégique au sein du Campus Provence Verte, jouant un rôle essentiel dans le bon déroulement des activités pédagogiques, administratives et opérationnelles de l'établissement. Il garantit la disponibilité, la performance et la sécurité de l'ensemble des infrastructures numériques, veillant ainsi à offrir un environnement de travail stable et sécurisé aux personnels enseignants, administratifs ainsi qu'aux étudiants.

Ce service est constitué d'une équipe restreinte mais expérimentée, sur laquelle repose l'ensemble du réseau et du parc informatique de l'établissement. Il est composé de deux personnes à temps plein: Monsieur Ioane Ducresson, mon tuteur de stage, qui cumule plusieurs années à la tête du service informatique tout en enseignant la physique et les sciences numériques et technologiques (NSI). Il est principalement chargé de l'administration des serveurs, de la gestion des systèmes et des aspects logiciels de l'infrastructure, de la maintenance des postes utilisateurs, de l'administration des réseaux, de la sécurisation des données et de la mise en place des stratégies de sauvegarde. À ses côtés, Monsieur Alexandre Perrier, fort de plus de vingt ans d'expérience au sein du campus, se consacre davantage au volet matériel: maintenance, dépannage, reconditionnement et mise en service des postes informatiques. Il répond également aux besoins quotidiens, aux demandes d'assistance et intervient rapidement en cas d'incidents, qu'il s'agisse de pannes matérielles, de problèmes de connexion réseau ou de dysfonctionnements logiciels. La complémentarité de leurs compétences, l'un plus orienté software et l'autre plus hardware, permet d'assurer à la fois la planification à long terme et la réactivité face aux aléas quotidiens. Elle garantit ainsi un fonctionnement optimal des outils numériques, tant pour le personnel que pour les élèves.

L'infrastructure informatique de l'établissement repose sur une architecture robuste comprenant plusieurs serveurs, tant physiques que virtualisés, ainsi qu'un parc important de postes fixes et mobiles répartis dans les salles de classe, les bureaux administratifs et les espaces communs. Le réseau et la protection des systèmes est assurée par une combinaison de solutions matérielles et logicielles comprenant pare-feux, systèmes de filtrage, contrôle des accès et audits réguliers. Le service administre également l'accès à Internet, les points d'accès Wi-Fi disséminés sur le campus, les imprimantes en réseau, la téléphonie IP ainsi que les dispositifs de sauvegarde centralisés (voir p.5 de l'annexe pour le plan du réseau).

Cependant, l'infrastructure actuelle, bien que opérationnelle, repose en grande partie sur des équipements anciens, qui nécessitent une maintenance soutenue pour garantir leur fiabilité.

Dans ce contexte, j'ai été amené, au cours de mon stage, à contribuer à la maintenance de cette infrastructure, en intervenant notamment sur des problématiques liées aux serveurs, aux postes de travail et à la connectivité réseau. Cette implication m'a permis de mieux comprendre les contraintes liées à l'exploitation d'un système vieillissant.

Conscients des limites de cette situation, les responsables du service informatique ont engagé une réflexion autour d'un projet de renouvellement global de l'infrastructure. Celui-ci vise à doter le campus d'un système d'information plus moderne, évolutif et sécurisé. La future architecture comprendra notamment des équipements réseau récents, une segmentation réseau plus fine grâce aux VLANs (Virtual Local Area Network, ou réseau local virtuel), ainsi que l'intégration de solutions de supervision et d'administration centralisées. Cette transition technologique permettra de répondre plus efficacement aux exigences actuelles en matière de performance, de sécurité et de gestion des ressources numériques.

Dans ce cadre, mon stage m'a permis de m'immerger concrètement au sein d'un environnement informatique riche, exigeant et en perpétuelle évolution. En intégrant pleinement le service, j'ai été amené à participer à de nombreuses interventions techniques, relevant à la fois de la maintenance curative, c'est-à-dire la gestion des incidents, résolution de pannes, assistance aux utilisateurs, etc... et de la maintenance préventive, donc tout ce qui tourne autour de l'amélioration des dispositifs de sécurité, la mise à jour des systèmes, les automatisations de tâches, etc... Ces expériences m'ont permis de développer des compétences opérationnelles solides, directement liées aux enjeux du stage, et d'appréhender la réalité du fonctionnement quotidien d'un service informatique dans un établissement de taille moyenne.

Dans les pages qui suivent, je présenterai en détail l'ensemble des missions qui m'ont été confiées, en commençant par la sécurisation des serveurs, un enjeu central pour le bon fonctionnement du campus.

3 Présentation des missions réalisées

3.1 Sécurisation des serveurs

3.1.1 Contexte

Dans le cadre de mon stage au sein du service informatique du Campus Provence Verte, l'une des premières missions majeures qui m'a été confiée a porté sur la sécurisation de l'ensemble des serveurs principaux de l'établissement. Ces serveurs représentent le cœur névralgique de l'infrastructure numérique du campus: ils assurent des fonctions critiques telles que l'authentification centralisée des utilisateurs, l'hébergement de services essentiels (tels que les bases de données, les services DNS (Domain Name Server, ou serveur de nom de domaine), DHCP (Dynamic Host Configuration Protocol, ou protocole de configuration dynamique d'hôte), Samba, etc...), la gestion des postes clients et la coordination des communications inter-systèmes au sein du réseau local.

Leur disponibilité, leur fiabilité et leur sécurité sont donc primordiales pour garantir la continuité des activités pédagogiques et administratives de l'établissement. Une défaillance ou une compromission des serveurs pourrait impacter de nombreux utilisateurs, tels que des enseignants, élèves ou personnel administratif, et compromettre la confidentialité, l'intégrité ou la disponibilité des données.

L'infrastructure s'appuie principalement sur deux serveurs virtualisés tournant sous Debian Linux:

- SE4AD, le contrôleur de domaine principal, repose sur la distribution Debian 12 Bookworm (il était précédemment sous Debian 11 Bullseye, la mise à jour ayant été réalisée durant mon stage). Ce serveur utilise SambaEdu pour centraliser la gestion des utilisateurs, des groupes

et des postes clients. Il joue également un rôle clé dans la définition des politiques de sécurité et d'accès, tout en hébergeant des services complémentaires comme MariaDB, Apache2, et en assurant la fonction de serveur DNS principal pour l'ensemble du parc informatique. Il convient toutefois de noter que d'autres serveurs DNS sont également présents dans l'infrastructure.

- SE4FS, également mis à jour vers Debian 12 Bookworm, constitue le serveur de fichiers et de services réseau. Il héberge les partages Samba accessibles aux différents utilisateurs selon leur profil, fournit le service DHCP permettant l'attribution automatique des adresses IP, et supporte des services comme Apache2, FreeRADIUS, PHP, ainsi que plusieurs protocoles de communication réseau, notamment NFS (non-pas Need For Speed, mais Network File System), et TFTP (Trivial File Transfert Protocol).

En complément de ces deux machines centrales, l'infrastructure comprend également d'autres serveurs spécifiques:

- Un serveur DNS AdGuard, qui tourne encore sous Debian 11 Bullseye, et qui est utilisé comme filtre DNS pour bloquer les publicités et certaines requêtes malveillantes.
- Un Windows Server nommé Charlemagne, est dédié à la gestion des documents administratifs du campus. Il fonctionne indépendamment de l'infrastructure Linux décrite ci-dessus.

L'ensemble des serveurs cités (à l'exception de Charlemagne) sont hébergés sur une infrastructure de virtualisation basée sur Proxmox VE (Virtual Environment, ou environnement virtuel), installée sur une machine physique (nouveau serveur Proxmox VE, Figure 1) équipée de disques SSD en RAID 1 (voir l'annexe p.5 pour plus de détails sur les composants et la configuration). Cette solution permet une gestion souple, centralisée et sécurisée des différentes machines virtuelles, tout en facilitant la maintenance, la sauvegarde et la réallocation des ressources matérielles.



Figure 1: Serveur Proxmox VE

Lorsque j'ai commencé à intervenir sur les serveurs de l'infrastructure, j'ai rapidement constaté que certaines mesures de sécurité élémentaires étaient déjà présentes, comme la restriction de certains accès ou la limitation des droits des utilisateurs, mais beaucoup étaient manquantes, et ont donc fait partie des tâches que j'ai effectuées. Ce processus de sécurisation ne s'est pas limité à une simple application de consignes. Il a impliqué une réelle démarche de veille technologique et de recherche documentaire, afin de mieux comprendre le rôle et le fonctionnement des services en place (SambaEdu, MariaDB, FreeRADIUS, Apache2, etc...), d'identifier les risques potentiels, et de déterminer les meilleures pratiques à mettre en œuvre

selon le contexte. J'ai consulté des documentations officielles, des retours d'expérience issus de la communauté open-source, ainsi que des recommandations de l'ANSSI, pour aligner les configurations sur des standards fiables et éprouvés.

Cette phase a également été marquée par une forte collaboration avec mon tuteur de stage, qui m'a guidé sur les enjeux liés à la sécurité du système d'information dans son ensemble. Ensemble, nous avons analysé les priorités, testé différentes approches, et validé chaque modification apportée, que ce soit l'activation du pare-feu (UFW, Uncomplicated FireWall, ou pare-feu simple), la sécurisation de l'accès SSH (Secure SHell, ou terminal sécurisé) par clé, ou encore la configuration précise de Fail2ban. Ce travail d'équipe a été essentiel pour intégrer de manière fluide les nouvelles configurations sans perturber l'usage quotidien des serveurs, surtout dans le contexte de préparation du baccalauréat, tout en assurant un renforcement progressif et maîtrisé de la sécurité du système.

Mais, avant d'apporter la moindre modification sensible aux serveurs virtuels de l'infrastructure, j'ai adopté une stratégie systématique de sauvegarde en créant des clones complets de chaque machine sur Proxmox VE (Figure 2). Ces clones agissent comme des points de restauration fiables, permettant de revenir rapidement à un état stable en cas de mauvaise manipulation, d'erreur de configuration ou d'interruption de service non prévue.

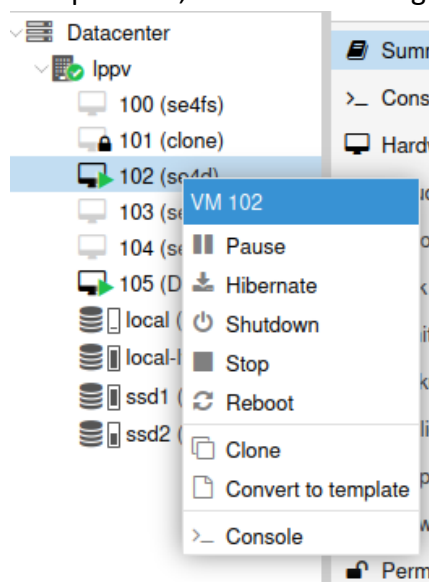


Figure 2: Création de clones

Cette approche m'a permis d'expérimenter librement des configurations complexes (pare-feu, services réseau, authentification Kerberos, etc...) sans crainte de compromettre définitivement la VM. Elle s'est révélée cruciale lors du diagnostic et de la résolution de problèmes réseau, notamment dans le cadre de Samba et de la gestion des flux LDAP/DNS entre postes clients et le contrôleur de domaine.

3.1.2 Plus de détail sur les outils et technologies utilisés

Avant d'aborder les tâches de sécurisation réalisées durant mon stage, il est essentiel de présenter les principaux outils, services et technologies sur lesquels repose l'infrastructure du Campus Provence Verte. Leur compréhension est très importante pour saisir la nature des opérations effectuées.

Le système d'exploitation utilisé sur la majorité des serveurs est Linux. C'est un système d'exploitation de type UNIX, libre et open source. À la différence de systèmes propriétaires comme Windows ou macOS, Linux peut être librement modifié, redistribué et adapté à différents usages. Il est très utilisé dans le monde des serveurs pour plusieurs raisons: sa stabilité, sa sécurité, sa

légèreté, sa modularité et le contrôle précis qu'il offre à l'administrateur système. Il permet de gérer efficacement les ressources, les services réseau, les utilisateurs, et propose un terminal très puissant. Linux est disponible sous forme de nombreuses distributions (ou "distros"), c'est-à-dire des variantes construites autour du même noyau mais avec des outils, gestionnaires de paquets et objectifs différents. Parmi les plus populaires figurent Ubuntu, Fedora, Arch Linux... et Debian, la distribution utilisée au sein de l'infrastructure du Campus Provence Verte.

Debian est une distribution connue pour sa fiabilité, et sa grande communauté. Elle est largement utilisée dans les environnements serveurs, notamment dans les établissements publics et les entreprises. Debian offre des mises à jour régulières, un grand nombre de logiciels disponibles via son gestionnaire de paquets aptitude (plus connu comme la commande "apt"), et une très bonne documentation.

Durant mon stage, j'ai eu l'occasion de travailler sur plusieurs serveurs basés sur le système d'exploitation Debian Linux. Deux versions de cette distribution ont été utilisées:

- Debian 11 "Bullseye", qui était installée au début de mon stage sur les serveurs SE4AD et SE4FS
- Debian 12 "Bookworm", la dernière version stable, que j'ai mise en place dans le cadre des mises à jour majeures effectuées sur ces mêmes machines.

Cette migration vers une version plus récente faisait partie des actions de sécurisation globales, dans le but d'améliorer la stabilité et la sécurité de l'infrastructure.

Avant de détailler plus les services et outils utilisés, il est essentiel de rappeler ce que l'on entend par "serveur". Dans l'imaginaire collectif, un serveur est souvent représenté comme une grande salle remplie de baies et de câbles. En réalité, un serveur est simplement un ordinateur, physique ou virtuel, conçu pour fournir des services ou des ressources à d'autres appareils d'un réseau, que l'on appelle des clients. Contrairement aux postes de travail classiques, les serveurs sont souvent optimisés pour fonctionner en continu, avec une tolérance accrue aux pannes, et sont souvent dédiés à des tâches spécifiques telles que l'hébergement d'un site web, le stockage de fichiers partagés ou encore la gestion de l'authentification des utilisateurs.

Les rôles que peut remplir un serveur dans une infrastructure sont multiples et complémentaires. Il ne s'agit pas seulement d'une machine répondant à une requête: un serveur est souvent au cœur des communications, de la gestion et de la sécurité d'un réseau informatique. Parmi ses nombreuses fonctions, un serveur peut tout d'abord assurer l'authentification des utilisateurs grâce à un service d'annuaire tel qu'un contrôleur de domaine, permettant une gestion centralisée des comptes et des droits d'accès. Il peut aussi jouer le rôle de serveur DHCP, attribuant automatiquement une adresse IP à chaque poste qui se connecte au réseau, ce qui évite d'avoir à effectuer une configuration manuelle poste par poste, souvent source d'erreurs ou de conflits d'adresses. Un autre rôle fondamental est celui de serveur DNS, chargé de traduire les noms de domaine en adresses IP. Cette fonction indispensable, rend possible une navigation fluide dans un réseau local comme sur Internet: sans DNS, il faudrait retenir les adresses IP de chaque service ou site à contacter, ce qui serait impraticable. Les serveurs peuvent également être configurés pour héberger des sites ou des applications web via un serveur HTTP (Hyper Text Transfer Protocol, ou protocole de transfert hypertexte) comme Apache2, l'un des plus utilisés dans le monde. Ce type de service permet par exemple d'accéder à des interfaces de gestion, des portails internes ou des plateformes web.

Dans un cadre plus collaboratif ou éducatif, un serveur peut fournir des partages de fichiers en réseau, accessibles par différents utilisateurs selon des droits définis. Ce rôle est notamment rempli par Samba, un outil de partage de fichiers compatible avec les environnements Windows, et sa déclinaison SambaEdu, conçue par des professeurs, pour les établissements scolaires. Ce dernier permet en plus la centralisation des utilisateurs, la gestion fine des permissions et une intégration

facilitée dans des environnements pédagogiques. D'autres services essentiels viennent compléter cette architecture. Un serveur peut héberger une base de données grâce à un moteur comme MariaDB, qui permet de stocker, organiser et interroger des données de manière rapide et structurée. Ce type de base est souvent utilisé en conjonction avec des applications web ou des systèmes d'authentification.

Enfin, un serveur joue un rôle crucial dans la sécurisation des connexions et des accès. Pour cela, plusieurs outils sont mis en œuvre, chacun couvrant un aspect spécifique de la sécurité. Le protocole SSH permet par exemple d'administrer les serveurs à distance de manière sécurisée grâce au chiffrement des échanges, évitant toute interception malveillante. Pour compléter cette protection, l'outil Fail2ban surveille les tentatives de connexion répétées: en cas de comportements suspects, comme des mots de passe erronés à répétition, il bloque temporairement les adresses IP à l'origine de ces tentatives.

Autre élément essentiel de la sécurité réseau: le pare-feu. Celui-ci agit comme un filtre, autorisant ou bloquant certains flux de données selon des règles précises. Dans le cadre de mon stage, c'est l'utilitaire UFW qui a été utilisé. Il simplifie grandement la gestion des règles de filtrage en s'appuyant sur le puissant système iptables, tout en le rendant plus accessible. Enfin, pour sécuriser l'accès aux services réseau, un serveur peut intégrer un système d'authentification centralisée tel que FreeRADIUS, souvent utilisé pour valider l'identité des utilisateurs avant de leur donner accès à des ressources critiques. Ces outils, bien que différents, ont un objectif commun: garantir la fiabilité, la confidentialité et l'intégrité des communications au sein du réseau.

L'ensemble de l'infrastructure que j'ai administrée repose sur une architecture virtualisée, déployée à l'aide de Proxmox VE (Virtual Environment), un hyperviseur libre, open source, basé sur Debian. Proxmox VE permet la gestion centralisée de machines virtuelles (VMs) et de conteneurs (LXC, LinuX Containers), avec une interface web très intuitive, des outils puissants de sauvegarde/restauration, et une prise en charge native des technologies de haute disponibilité, de clustering et de stockage partagé.

Un hyperviseur, comme Proxmox VE, est un logiciel qui permet de faire fonctionner plusieurs systèmes d'exploitation (appelés "machines virtuelles") sur un même serveur physique. Cela permet:

- Une réduction des coûts matériels, car plusieurs services peuvent tourner sur une seule machine physique.
- Une meilleure isolation, chaque VM étant indépendante des autres, c'est-à-dire qu'un crash du serveur de fichiers n'impactera pas le contrôleur de domaine.
- Une grande souplesse d'administration, puisqu'il est possible de démarrer, arrêter, cloner ou sauvegarder chaque machine virtuelle individuellement.

Autrement dit, un seul serveur physique peut héberger plusieurs machines virtuelles, chacune jouant un rôle distinct: serveur DNS, contrôleur de domaine, serveur de fichiers, etc...

Dans mon cas, presque tous les serveurs que j'ai administrés étaient des machines virtuelles tournant sur un hôte physique sous Proxmox VE. Ce type d'architecture m'a permis de comprendre les enjeux liés à la virtualisation, à la gestion des ressources partagées, et à la maintenance d'une infrastructure modulaire, où chaque composant peut être surveillé, mis à jour ou remplacé indépendamment des autres.

3.1.3 Tâches effectués

Plusieurs points critiques étaient à traiter. Par exemple, aucun pare-feu n'était activé, ce qui laissait de nombreux ports ouverts inutilement, augmentant ainsi la surface d'attaque du système. De plus, certains services tournaient encore avec des configurations par défaut ou des versions obsolètes, ce qui exposait les machines à des vulnérabilités connues et documentées. Il devenait

donc nécessaire d'entreprendre un travail approfondi de durcissement, aussi bien au niveau des services que de l'architecture globale. La sécurisation s'est donc articulée autour de plusieurs axes:

- Configuration d'un pare-feu ufw personnalisé sur chaque serveur, afin de filtrer efficacement les connexions entrantes et sortantes en fonction des besoins des serveurs.
- Mise en place de systèmes de détection et de prévention des intrusions, via des outils comme Fail2ban, pour bloquer automatiquement les tentatives d'accès non autorisées ou répétées.
- Sécurisation des échanges distants, en forçant l'utilisation de protocoles sécurisés et en désactivant les services obsolètes ou non nécessaires.
- Correction des erreurs de configuration existantes, notamment sur la gestion des ports ouverts de certains services.
- Durcissement des services critiques, comme SSH, Samba, MariaDB, Apache2 ou encore FreeRADIUS, afin de limiter la surface d'attaque et de réduire les risques d'exploitation.
- Mise à jour des systèmes d'exploitation, pour passer de Debian 11 (Bullseye) à Debian 12 (Bookworm), étape essentielle pour bénéficier des dernières corrections de sécurité.

L'approche adoptée a été progressive. Elle a débuté par un état des lieux précis de l'existant: services actifs, ports ouverts, utilisateurs et groupes, règles de pare-feu en place, journaux systèmes, vulnérabilités connues. Cela m'a permis de prioriser les actions selon leur criticité.

Avant toute action de sécurisation, une première analyse du serveur SE4AD a été menée afin d'identifier les services exposés, les ports ouverts, et les processus actifs. Pour cela, la commande suivante a été utilisée (Figure 3, voir l'annexe p.6 pour le résultat complet):

```
netstat -laputen
```

Cette commande permet d'afficher les connexions réseau actives, les ports d'écoute, ainsi que les programmes qui les utilisent. L'option -l signifie "listening", c'est-à-dire que seuls les ports à l'écoute seront affichés. L'option -a affiche toutes les connexions et tous les ports d'écoute. -p affiche le nom du programme (et le PID, Process IDentifier, ou identifiant du processus) responsable de chaque connexion, ce qui est essentiel pour repérer quel service écoute sur quel port. Les options -u et -t précisent respectivement les connexions UDP et TCP. Enfin, -e donne des informations supplémentaires comme les files d'attente, et -n évite la résolution des noms de domaine et affiche les adresses IP numériques, ce qui accélère l'exécution et améliore la lisibilité.

```
root@se4ad:~# netstat -laputen
Connexions Internet actives (serveurs et établies)
Proto Recv-Q Send-Q Adresse locale Adresse distante Etat Utilisatr Inode PID/Program name
tcp 0 0 0.0.0.0:636 0.0.0.0:* LISTEN 0 11809 372/samba: task[lda
tcp 0 0 0.0.0.0:445 0.0.0.0:* LISTEN 0 11905 371/smbd
tcp 0 0 0.0.0.0:49152 0.0.0.0:* LISTEN 0 11587 364/samba: task[rpc
tcp 0 0 0.0.0.0:49153 0.0.0.0:* LISTEN 0 11693 392/samba: task[rpc
tcp 0 0 0.0.0.0:49154 0.0.0.0:* LISTEN 0 11700 392/samba: task[rpc
tcp 0 0 0.0.0.0:3268 0.0.0.0:* LISTEN 0 11810 372/samba: task[lda
tcp 0 0 0.0.0.0:3269 0.0.0.0:* LISTEN 0 11811 372/samba: task[lda
```

Figure 3: Résultat partiel de la commande netstat -laputen

Grâce à cette commande, il a été possible de constater que plusieurs ports (séparés de l'IP par deux points: "IP:port") critiques étaient ouverts, notamment ceux liés à SSH (port 22 par défaut), Samba (ports 137, 138, 139 et 445), LDAP (Lightweight Directory Access Protocol, ou protocole d'accès léger aux annuaires), DNS, Kerberos, entre autres. Ces ports étant indispensables au fonctionnement du contrôleur de domaine Samba AD, ils ne peuvent pas être fermés, mais il est crucial de restreindre leur accessibilité au strict nécessaire. Parallèlement, une vérification des services en cours d'exécution a été réalisée avec la commande suivante (Figure 4, voir l'annexe p.6 pour le résultat complet):

```
systemctl list-units --type=service --state=running
```

Cette commande interroge systemd pour lister tous les services actifs (--type=service) dont l'état est "running" (--state=running). Cela a permis de recenser les services essentiels au fonctionnement du

serveur comme samba-ad-dc, ssh, fail2ban, rsyslog, mais aussi de s'assurer qu'aucun service inutile ou suspect ne tournait en tâche de fond.

```
root@se4ad:~# systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
ntp.service                        loaded active running Network Time Service
rsyslog.service                   loaded active running System Logging Service
samba-ad-dc.service               loaded active running Samba AD Daemon
ssh.service                       loaded active running OpenBSD Secure Shell server
```

Figure 4: Résultat partiel de la commande `systemctl list-units --type=service --state=running`

Une fois ce diagnostic posé, l'activation d'un pare-feu est apparue indispensable. Le serveur n'en disposait pas initialement, le rendant particulièrement vulnérable. Le pare-feu UFW a été installé en utilisant la commande `"apt install ufw -y"` où l'option `-y` permet de valider automatiquement l'installation sans demander confirmation à l'utilisateur, et a été choisi pour sa simplicité et sa compatibilité avec les environnements Debian.

La première étape a consisté à définir des règles de base avec les commandes suivantes:

```
ufw default deny incoming
ufw default allow outgoing
```

La première commande bloque toutes les connexions entrantes par défaut, ce qui signifie que tout trafic entrant non explicitement autorisé sera refusé. La seconde permet au serveur d'initier librement des connexions sortantes, par exemple pour mettre à jour les paquets, synchroniser l'heure, ou envoyer des logs. Le reste de sa configuration se fera plus tard, car il m'est apparu nécessaire de faire un travail de durcissement de ces services critiques.

D'autres réglages ont été appliqués dans le fichier de configuration `"/etc/ssh/sshd_config"` pour renforcer davantage la sécurité de l'accès SSH. Ces directives permettent de mieux contrôler le comportement des sessions SSH et de limiter les possibilités d'exploitation par un attaquant:

- La directive `"LoginGraceTime"` a été réduite à 1 minute (au lieu de la valeur par défaut de 2 minutes). Ce paramètre définit le temps maximal accordé à un utilisateur pour s'authentifier après avoir établi une connexion SSH. Passé ce délai sans authentification réussie, le serveur ferme automatiquement la session. Réduire ce temps permet de raccourcir la fenêtre durant laquelle un attaquant pourrait essayer de deviner un mot de passe.
- `"MaxAuthTries"` a été fixée à 3, ce qui limite à trois le nombre de tentatives de mot de passe autorisées par session. Au-delà, la connexion est immédiatement interrompue. Cela empêche les attaques par force brute qui essaieraient plusieurs combinaisons de mots de passe successivement.
- Ensuite, `"MaxSessions"` a été fixé à 4, ce qui limite le nombre de sessions simultanées pouvant être ouvertes par une seule connexion SSH. Par défaut, ce nombre est plus élevé (10), ce qui peut être exploité pour saturer les ressources ou contourner certaines restrictions. Réduire ce nombre contribue à limiter les abus tout en permettant une utilisation normale.
- Enfin, la directive `"PermitRootLogin"` est restée sur la valeur `yes` dans ce contexte de test, ce qui signifie que l'utilisateur `root` peut se connecter directement via SSH. Ce paramètre représente une faille potentielle en production, car les attaques ciblent souvent le compte `root`. Il est donc fortement recommandé de le désactiver (`"PermitRootLogin no"`) et de préférer une connexion via un utilisateur non privilégié, puis une élévation de privilèges avec `sudo`.

Une fois ces réglages appliqués, le service SSH a été redémarré avec la commande `"systemctl restart ssh"` pour prendre en compte les modifications.

En complément des réglages appliqués au service SSH, le service Fail2ban a été installé afin d'automatiser la détection et le blocage des tentatives de connexion infructueuses. Fail2ban agit comme une protection dynamique: il surveille les journaux du système, en particulier le fichier `"/var/log/auth.log"` dans le cas du service SSH, et déclenche automatiquement des bannissements temporaires pour toute adresse IP manifestant un comportement suspect, comme des tentatives de connexion répétées avec des identifiants erronés. L'installation du service a été réalisée via la commande `"apt install fail2ban -y"`, et pour activer immédiatement le service dès son installation, la ligne de commande `"systemctl enable --now fail2ban"` a été utilisée: elle active le démarrage automatique du service au démarrage du système (enable) et s'exécute immédiatement (--now).

Une configuration personnalisée a ensuite été créée dans le fichier `"/etc/fail2ban/jail.d/ssh6602.conf"`, spécifiquement pour le service SSH. Cette configuration définit trois paramètres principaux:

- Le paramètre `"maxretry = 3"` indique que si une adresse IP échoue à s'authentifier trois fois, elle sera considérée comme suspecte.
- La directive `"findtime = 10m"` précise que ces trois échecs doivent avoir lieu dans une fenêtre de dix minutes, sinon le compteur est réinitialisé.
- Enfin, `"bantime = 1h"` définit la durée de blocage, ici fixée à une heure complète. Cette mesure simple mais efficace permet de bloquer les tentatives de brute-force en réduisant fortement la cadence possible des essais.

Le service `samba-ad-dc`, qui remplit le rôle de contrôleur de domaine Active Directory dans une infrastructure basée sur Samba, représente un point névralgique de l'architecture réseau. Il expose plusieurs services essentiels tels que **SMB** (pour le partage de fichiers), LDAP (pour l'annuaire d'authentification), DNS (pour la résolution de noms) et Kerberos, chacun fonctionnant sur des ports réseau spécifiques qui doivent rester ouverts pour permettre aux machines clientes de joindre le domaine, de s'authentifier et de localiser les ressources nécessaires.

Parmi ces services, Kerberos joue un rôle fondamental dans la gestion de l'authentification sécurisée. Il repose sur un mécanisme de tickets qui permet aux utilisateurs et aux services de s'authentifier mutuellement sans transmettre de mots de passe en clair sur le réseau. Lorsqu'un utilisateur se connecte à une machine membre du domaine, il contacte le serveur Kerberos, intégré au contrôleur de domaine Samba, pour obtenir un TGT (Ticket-Granting Ticket, ou ticket d'octroi de ticket). Ce TGT prouve que l'utilisateur a bien été authentifié et peut ensuite être utilisé pour obtenir des tickets de service permettant d'accéder aux ressources du réseau (partages, imprimantes, services, etc...), toujours sans renvoyer son mot de passe.

Kerberos fonctionne principalement sur le port UDP 88 (et parfois TCP 88), et nécessite une synchronisation horaire précise entre les machines du réseau, car les tickets sont horodatés pour éviter toute tentative de réutilisation malveillante. Cette exposition de multiples services, bien que nécessaire au bon fonctionnement du domaine, augmente mécaniquement la surface d'attaque si Samba est mal configuré. Notamment, certaines failles peuvent apparaître si des connexions anonymes ou invitées sont autorisées. Pour réduire cette surface, plusieurs directives de configuration ont été appliquées dans le fichier `"/etc/samba/smb.conf"`, dans le but de désactiver les accès non authentifiés et de renforcer le contrôle d'accès, comme expliqué en détail ci-dessous:

- La directive `"map to guest = never"` a été définie afin d'interdire toute tentative de connexion anonyme d'être automatiquement convertie en une session "invité". Cela empêche un utilisateur non authentifié d'accéder au réseau de manière implicite.
- Ensuite, la ligne `"restrict anonymous = 2"` bloque les requêtes LDAP anonymes, ce qui protège contre les tentatives de collecte d'informations sur le domaine sans authentification.

- Enfin, "guest ok =" ne désactive explicitement toute autorisation d'accès invité sur les partages Samba.

Ces trois réglages agissent ensemble pour assurer que seuls les utilisateurs authentifiés puissent interagir avec les ressources du domaine. Une fois les modifications apportées, le service Samba a été redémarré à l'aide de la commande "systemctl restart samba-ad-dc", ce qui permet de recharger la configuration.

Le service rsyslog, chargé de la journalisation des événements système, occupe une place centrale dans l'architecture de sécurité d'un serveur. Il enregistre automatiquement une multitude d'activités critiques, notamment les connexions SSH, les actions des utilisateurs, le comportement des services comme sshd, samba-ad-dc ou encore les erreurs système générées par le noyau. Ces fichiers de log constituent une source précieuse d'information dans le cadre d'une supervision quotidienne, mais surtout en cas d'analyse post-incident, lorsqu'il s'agit de retracer précisément la chronologie d'une attaque, d'un dysfonctionnement ou d'une élévation de privilèges. Toutefois, sans précaution particulière, ces mêmes fichiers peuvent devenir une faiblesse: ils peuvent être altérés, lus ou effacés par un attaquant s'il parvient à obtenir un accès non autorisé.

Pour renforcer la sécurité autour de ces journaux, une première mesure a consisté à restreindre strictement leurs permissions. La commande "chmod -R go-rwx /var/log" a été appliquée. Elle retire, de manière récursive (-R), tous les droits d'accès (r pour lecture, w pour écriture, x pour exécution) aux utilisateurs du groupe (g) et aux autres utilisateurs non privilégiés (o). Seul l'utilisateur propriétaire reste autorisé à accéder aux fichiers. Cette opération permet de s'assurer qu'aucun compte non root, même appartenant à un groupe système, ne puisse lire ou manipuler le contenu du répertoire "/var/log".

En complément, la commande "chown -R root:root /var/log" a permis de réaffecter la propriété de tous les fichiers et sous-dossiers du répertoire "/var/log" à l'utilisateur root et au groupe root. Cela garantit que seul le compte administrateur du système dispose des droits nécessaires pour accéder ou modifier ces fichiers. Par défaut, rsyslog peut également être configuré pour accepter des journaux envoyés à distance, via les protocoles UDP ou TCP, sur le port 514. Cette fonctionnalité est couramment utilisée dans des environnements où une centralisation des logs est mise en place, notamment pour agréger les journaux de plusieurs serveurs vers un collecteur centralisé. Cependant, dans le contexte de ce serveur, aucune écoute réseau n'est nécessaire. Pour désactiver cette possibilité, les modules d'entrée correspondants dans le fichier de configuration "/etc/rsyslog.conf" ont été laissés commentés. Plus précisément, les lignes "#module(load="imudp")" et "#input(type="imudp" port="514")" pour UDP, ainsi que "#module(load="imtcp")" et "#input(type="imtcp" port="514")" pour TCP, n'ont pas été décommentées. Cette configuration empêche toute réception de journaux via le réseau, ce qui élimine certains vecteurs d'attaque tels que les dénis de service par saturation de logs ou l'injection de fausses entrées destinées à camoufler des activités malveillantes.

Pour compléter ce dispositif de sécurisation, une solution d'audit en temps réel a été ajoutée grâce à l'installation du paquet auditd. Ce démon permet d'enregistrer, de manière fiable, toute tentative d'accès à des fichiers sensibles, y compris les journaux systèmes. Une règle spécifique a été appliquée à l'aide de la commande "auditctl -w /var/log/ -p wa -k log_watch". Le paramètre "-w /var/log/" ordonne à auditd de surveiller le répertoire des logs, tandis que "-p wa" indique que l'on souhaite enregistrer les opérations d'écriture (w) ainsi que les modifications d'attributs de fichiers (a), comme un changement de droits ou de propriété. L'option "-k log_watch" ajoute une étiquette personnalisée à cette règle, ce qui facilite la recherche ultérieure dans les journaux d'audit. Pour consulter les événements enregistrés par cette règle, la commande "ausearch -k log_watch" permet d'extraire précisément les entrées liées à ce tag. Ainsi, toute tentative légitime ou suspecte de

modification des fichiers de log est consignée, rendant beaucoup plus difficile une falsification discrète des traces.

Enfin, la présence du système de confinement AppArmor a été vérifiée via la commande "aa-status". Cette commande permet de connaître l'état des profils de sécurité actifs, qu'ils soient en mode appliqué ou permissif, ainsi que les processus actuellement protégés.

Dans cette logique de défense en profondeur, un audit de sécurité complet a également été mené sur les deux serveurs SE4AD et SE4FS, à l'aide de l'outil Lynis. Ce dernier est un auditeur open source conçu pour les systèmes Linux, capable d'identifier les vulnérabilités, les configurations faibles, ou les écarts par rapport aux bonnes pratiques de sécurité. Son installation a été réalisée via la commande "apt install lynis", puis un audit complet du système a été lancé avec "lynis audit system" (Figure 5).

```
Lynis security scan details:

Hardening index : 67 [##### ]
Tests performed : 272
Plugins enabled : 1

Components:
- Firewall [V]
- Malware scanner [X]
```

Figure 5: Résultat partiel de la commande lynis audit system

Les résultats obtenus ont été analysés pour chaque serveur. Lynis fournit une synthèse claire des points à améliorer, classés selon leur niveau de priorité. Suivant les résultats de l'audit de sécurité, un dispositif de détection des malwares a été mis en place sur les deux serveurs. Pour cela, l'antivirus ClamAV a été installé avec la commande "apt update -y && apt install clamav clamav-daemon -y". Ce logiciel open source est spécialisé dans la détection de virus, chevaux de Troie et autres logiciels malveillants sur les systèmes de fichiers. Avant de procéder au premier scan, la base de signatures virales a été mise à jour manuellement: le service clamav-freshclam, chargé de cette mise à jour, a d'abord été temporairement arrêté avec "systemctl stop clamav-freshclam", puis la commande "freshclam" a été exécutée pour récupérer les dernières définitions de menaces. Le service a ensuite été relancé via "systemctl start clamav-freshclam".

Un premier scan complet du système a alors été lancé avec la commande "clamscan -r / --bell -i", dans laquelle l'option -r active le mode récursif (analyse de tous les sous-dossiers), --bell déclenche un signal sonore en cas de détection, et -i permet d'afficher uniquement les fichiers jugés infectés, pour se concentrer sur les anomalies réelles. Toutefois, ce type de scan manuel ne peut pas être fiable à long terme sans automatisation. C'est pourquoi une tâche planifiée a été mise en place via crontab, de manière à lancer automatiquement un scan complet chaque dimanche à 23h50. La ligne suivante a été ajoutée au fichier de planification, accessible via la commande "crontab -e":

```
50 23 * * 0 clamscan -r / --bell -i >> /var/log/clamav/clamscan.log
```

Ainsi, tous les résultats sont enregistrés dans un fichier de log dédié, facilitant le suivi et l'analyse rétrospective.

Pour renforcer davantage la capacité de détection, un second outil, Linux Malware Detect (LMD), a été installé. Spécifiquement conçu pour les environnements Linux, notamment ceux hébergeant des sites web ou des scripts PHP (Hypertext Preprocessor), LMD dispose d'une base virale particulièrement adaptée aux menaces web modernes. Son installation a nécessité un téléchargement manuel depuis le site officiel. Après s'être placé dans le répertoire temporaire avec "cd /tmp", l'archive "maldetect-current.tar.gz" a été récupérée, puis extraite grâce aux commandes suivantes:

```
wget http://www.rfxn.com/downloads/maldetect-current.tar.gz
```

`tar -xzf maldetect-current.tar.gz`

Le répertoire résultant des commandes a été ouvert et le script d'installation se trouvant à l'intérieur est exécuté avec `./install.sh`, ce qui a permis d'intégrer LMD dans le système.

Un scan initial a été lancé avec `"maldet -a /"`, pour effectuer une analyse complète du système. Par défaut, LMD utilise sa propre base de données virale, mais il peut aussi être configuré pour tirer parti du moteur ClamAV, permettant ainsi une détection plus rapide et plus large. Pour activer cette intégration, le fichier de configuration `"/usr/local/maldetect/conf.maldet"` a été modifié afin de passer la directive `"scan_clamscan"` à 1. Cela active le couplage entre LMD et ClamAV, exploitant les bases virales des deux outils de manière synergique.

En prévision de l'automatisation des analyses, les fichiers de logs ont été créés manuellement à l'aide de la commande `"touch /var/log/clamav/clamscan.log /var/log/clamav/lmdscan.log"`, afin de s'assurer qu'ils existent avant toute tentative d'écriture par les outils de scan. Une fois ces fichiers en place, leur sécurité a été renforcée. Le propriétaire a été défini à root pour l'utilisateur et le groupe via la commande `"chown root:root"`, empêchant toute modification par des comptes non privilégiés.

Les permissions ont ensuite été restreintes avec `"chmod 644"`, ce qui correspond à un réglage précis des droits d'accès:

- Le 6 (en base octale) pour le propriétaire (root) signifie lecture (4) + écriture (2), donc `rw-`.
- Le 4 pour le groupe signifie lecture seule, donc `r--`.
- Le 4 pour les autres utilisateurs signifie également lecture seule, soit `r--`.

Ainsi, seul l'utilisateur root est autorisé à modifier le contenu de ces fichiers, tandis que les autres utilisateurs (y compris les services ou scripts non exécutés avec des privilèges élevés) peuvent, au mieux, en consulter le contenu. Cette restriction permet une surveillance ou une consultation sécurisée, tout en empêchant toute modification non autorisée. Cela garantit l'intégrité des journaux produits par les analyses antivirus, tout en conservant une visibilité en lecture pour les outils de supervision ou d'analyse. Grâce à cette configuration, l'ensemble du système peut être analysé efficacement avec une seule commande: `"maldet -a /"`. Cette commande tire parti des fonctionnalités combinées de LMD et ClamAV. En s'appuyant sur ces deux moteurs complémentaires, le système gagne en performance en évitant les doublons dans les analyses, tout en optimisant l'utilisation des ressources. De plus, cette approche renforce significativement la détection des menaces en croisant deux bases de signatures distinctes. Elle permet de mieux identifier les malwares ciblant les environnements Linux, ainsi que les scripts malveillants souvent utilisés sur les serveurs web, notamment ceux s'appuyant sur des technologies comme PHP.

Les journaux générés lors de ces scans sont centralisés et sécurisés, facilitant ainsi la traçabilité et l'analyse en cas d'incident. Ce dispositif agit comme une couche de défense essentielle au sein de la politique globale de sécurité, en limitant les risques de propagation d'attaques ou de fuites de données. Cependant, la sécurisation ne se limite pas à la simple mise en place d'outils antivirus. Une fois ces services correctement configurés et durcis individuellement, il est crucial de contrôler précisément les points d'entrée et les communications réseau des serveurs. C'est pourquoi une attention particulière a été portée à la gestion des ports ouverts et au filtrage des flux réseau en mettant en place un pare-feu strict, déployé en s'appuyant sur UFW, un outil simple mais puissant permettant de définir des règles claires sur le trafic entrant et sortant.

La configuration du pare-feu UFW sur le serveur SE4AD repose sur une politique de sécurité stricte, conçue pour bloquer toutes les connexions entrantes par défaut, tout en permettant les connexions sortantes. Cette approche garantit que le serveur ne reçoit aucune communication non sollicitée, réduisant ainsi considérablement le risque d'intrusions externes. Les règles qui ont été

définies autorisent uniquement des flux spécifiques provenant du réseau interne identifié par la plage d'adresses 126.0.0.0/8. Ces flux correspondent à des services indispensables au bon fonctionnement du serveur d'annuaire et d'authentification.

Par exemple, le port TCP 6602 est utilisé pour le service SSH. Le choix d'un port non standard, ici 6602 au lieu du port par défaut 22, a été fait pour limiter les tentatives de connexion non autorisées par des robots ou scripts automatisés qui balayent généralement les ports courants.

Les ports UDP 137 et 138 ainsi que le port TCP 139 sont associés au protocole NetBIOS (Network Basic Input/Output System). Ce protocole, utilisé dans les réseaux Windows, permet plusieurs fonctionnalités essentielles comme la découverte des machines sur le réseau, la résolution de noms NetBIOS (similaire au DNS, mais à l'échelle locale), et la gestion des sessions entre ordinateurs. Il est particulièrement utilisé par Samba, une implémentation libre des services de fichiers et d'impression compatibles Windows, souvent déployée sur des serveurs Linux.

Le port TCP 445, quant à lui, est lié au service Microsoft-DS (Directory Services), utilisé par le protocole SMB (Server Message Block) en version moderne. Ce protocole permet le partage de fichiers, de dossiers, d'imprimantes et d'autres ressources réseau entre ordinateurs. Contrairement à NetBIOS, SMB sur le port 445 fonctionne directement sans encapsulation dans NetBIOS, ce qui en fait une évolution plus efficace du partage de ressources dans les environnements Microsoft, et également exploitée par Samba dans les environnements Linux.

Le serveur SE4AD autorise également les connexions sur le port 53 en UDP, qui correspond au service DNS. Ce service est fondamental pour la résolution de noms de domaine en adresses IP, permettant ainsi aux machines du réseau de localiser les ressources à partir de noms lisibles, comme "serveur.ad.local", plutôt que d'utiliser des adresses numériques. Il est utilisé à la fois en tant que client DNS pour résoudre les noms extérieurs, et en tant que serveur DNS local pour résoudre les noms internes de l'infrastructure Active Directory.

En ce qui concerne l'annuaire, les ports 389 en TCP et 636 en TCP sont ouverts respectivement pour LDAP en clair, et LDAP sécurisé via TLS (LDAPS). LDAP est un protocole central dans les environnements Active Directory, car il permet de consulter et de modifier des informations dans l'annuaire: utilisateurs, groupes, ordinateurs, unités organisationnelles, stratégies de groupe, etc... Le port 389 est souvent utilisé pour les requêtes internes ou lors d'une première connexion avant le passage à une session sécurisée, tandis que le port 636 garantit un chiffrement dès l'établissement de la connexion.

Le protocole Kerberos, largement utilisé dans les environnements Windows pour l'authentification réseau, utilise plusieurs ports sur le serveur SE4AD. Le port 88 en TCP est utilisé pour l'émission de tickets d'authentification auprès du Key Distribution Center (KDC), qui est une fonction intégrée à Active Directory. Ce mécanisme d'authentification repose sur des tickets à durée limitée, permettant une sécurité renforcée sans nécessiter de transmettre le mot de passe à chaque connexion. Les ports 464 en TCP et UDP sont utilisés pour le changement ou la réinitialisation des mots de passe via Kerberos, notamment lorsque l'utilisateur modifie son mot de passe à distance ou qu'un administrateur effectue une réinitialisation.

Les ports 3268 et 3269 en TCP sont utilisés pour le Global Catalog. Le Global Catalog est un service spécifique à Active Directory qui permet une recherche rapide d'objets au sein de plusieurs domaines du même arbre ou de la même forêt. Le port 3268 transporte les requêtes non sécurisées, tandis que le port 3269 est utilisé pour les connexions chiffrées avec TLS. Ce service permet, par

exemple, à un utilisateur d'un domaine de retrouver un groupe situé dans un autre domaine, ce qui est indispensable dans une architecture multi-domaines.

Le port 3306 en TCP correspond à la base de données MariaDB, qui est une version libre et performante de MySQL. Cette base de données peut être utilisée pour stocker des informations d'authentification, des journaux d'activité, des configurations centralisées, ou encore des tables utilisées par certains services comme FreeRADIUS ou des interfaces web de gestion.

Pour assurer une bonne synchronisation des horloges réseau, le serveur SE4AD utilise le protocole NTP (Network Time Protocol) sur le port 123 en UDP. Ce service garantit que tous les équipements du réseau partagent une heure précise et cohérente, ce qui est crucial pour le bon fonctionnement de Kerberos, la validité des journaux (logs) et l'analyse des événements de sécurité.

Enfin, les ports 49152 à 49154 en TCP appartiennent à la plage dynamique des ports RPC (Remote Procedure Call, ou appel de procédure à distance) utilisés par les services Microsoft. Ces ports sont alloués dynamiquement par le système pour permettre aux applications distribuées de communiquer entre elles, par exemple lors d'opérations de gestion à distance via les consoles d'administration, ou lors de synchronisations entre contrôleurs de domaine.

Comme pour SE4AD, les premières étapes de sécurisation de SE4FS ont débuté par un état des lieux des services en écoute et en cours d'exécution, réalisé à l'aide des commandes "netstat -laputen" et "systemctl list-units --type=service --state=running", afin d'identifier les processus actifs, les ports ouverts et les programmes associés. Les résultats (voir l'annexe p.7 pour seulement le résultat complet de systemctl list-units --type=service --state=running, car le résultat de netstat fait 22 pages) ont permis d'ajuster finement les règles du pare-feu UFW. La même politique de blocage des connexions entrantes ("ufw default deny incoming") et d'autorisation des connexions sortantes ("ufw default allow outgoing") a été appliquée.

Le service Apache2, utilisé pour l'hébergement de ressources web, a été allégé en désactivant deux modules non essentiels. Cela a été effectué avec la commande "a2dismod status autoindex", qui désactive respectivement les modules "status", qui fournit une page d'état du serveur, potentiellement sensible, et "autoindex", qui génère automatiquement une liste des fichiers d'un répertoire lorsqu'aucune page d'index n'est présente. Supprimer ces modules réduit la surface d'exposition et évite de divulguer des informations non nécessaires. Afin de renforcer la sécurité active du serveur web contre les tentatives d'accès malveillantes, une protection par Fail2ban a été mise en place. Un fichier de configuration a été créé dans "/etc/fail2ban/jail.local" avec les paramètres suivants:

```
[apache]
enabled = true
port = http,https
filter = apache-auth
logpath = /var/log/apache2/*error.log
maxretry = 5
```

Cette configuration:

- Active la surveillance du service Apache ("enabled = true"),
- Spécifie les ports utilisés (port = http,https),
- Applique un filtre prédéfini destiné aux échecs d'authentification sur Apache ("filter = apache-auth"),
- Définit le chemin vers les journaux d'erreurs à analyser ("logpath"),
- Et fixe à cinq le nombre maximal de tentatives avant blocage ("maxretry = 5").

Ce mécanisme permet de bloquer temporairement les adresses IP responsables d'accès répétés non autorisés, réduisant le risque d'attaques par dictionnaire ou force brute.

Concernant le service MariaDB, deux mesures principales ont été mises en œuvre. Tout d'abord, la liaison réseau du serveur a été restreinte à une seule adresse IP en modifiant la directive `bind-address` dans le fichier de configuration `"/etc/mysql/mariadb.conf.d/50-server.cnf"` comme suit:

```
bind-address = 126.10.3.200
```

Cette modification empêche MariaDB d'écouter sur toutes les interfaces réseau, le limitant ainsi à l'adresse interne du serveur. Ensuite, pour appliquer les réglages de base en matière de sécurité, l'utilitaire interactif `"mysql_secure_installation"` a été exécuté. Il permet de supprimer les utilisateurs anonymes, désactiver les connexions root distantes, supprimer la base de test, et renforcer les mots de passe. La désactivation de l'accès root distant a également été vérifiée et renforcée en exécutant cette commande SQL directement depuis un terminal MySQL:

```
DELETE FROM mysql.user WHERE User='root' AND Host!='localhost';
```

Cette requête supprime toute entrée permettant à l'utilisateur root de se connecter depuis une machine distante, ne laissant que l'accès depuis localhost.

Le serveur FreeRADIUS a aussi été configuré pour restreindre les connexions entrantes uniquement aux clients autorisés. Pour cela, la configuration du fichier `"/etc/freeradius/3.0/clients.conf"` a été ajustée avec une déclaration de client:

```
client se4fs {
    ipaddr = 126.0.0.0/8
    secret = mdp
}
```

Cette directive limite l'accès au service aux seules machines appartenant au sous-réseau 126.0.0.0/8, avec un mot de passe partagé (secret) nécessaire à l'authentification.

Côté DHCP, deux mesures simples ont été prises. La directive `"deny unknown-clients;"` a été ajoutée à la configuration du serveur pour empêcher les machines non déclarées de recevoir une adresse IP, ce qui empêche un utilisateur inconnu ou malveillant de s'intégrer facilement au réseau. Ensuite, la plage d'adresses a été strictement définie dans le bloc subnet:

```
subnet 126.0.0.0 netmask 255.0.0.0 {
    range 126.10.12.1 126.10.17.255;
    option routers 126.10.0.1;
}
```

La sécurisation de PHP, utilisé conjointement avec Apache2 via le module PHP-FPM (FastCGI Process Manager, ou gestionnaire de processus FastCGI), a nécessité la modification de plusieurs directives critiques dans le fichier de configuration `"/etc/php/7.4/fpm/php.ini"`. Ces ajustements visent à limiter les risques d'exploitation via les scripts PHP en désactivant des fonctions potentiellement dangereuses et en évitant l'exposition d'informations sensibles. Les paramètres suivants ont été appliqués:

```
allow_url_fopen = Off
expose_php = Off
display_errors = Off
log_errors = On
```

- Le paramètre `"allow_url_fopen = Off"` désactive la possibilité pour PHP d'ouvrir des flux à distance à travers des fonctions telles que `fopen()`, `include()` ou encore `file_get_contents()`. Cela empêche efficacement les attaques par inclusion de fichier distant (Remote File Inclusion), un vecteur courant d'exécution de code malveillant à distance.
- La directive `"expose_php = Off"` supprime l'en-tête HTTP qui, par défaut, révèle la version exacte de PHP utilisée par le serveur. Cette information est inutile pour le fonctionnement

normal du site, mais précieuse pour un attaquant, notamment dans le cadre d'un scan de vulnérabilités ciblées sur une version précise.

- En mettant "display_errors = Off", les erreurs d'exécution ne sont plus affichées directement dans le navigateur. Cette mesure empêche la divulgation de détails internes en cas de dysfonctionnement, comme des chemins de fichiers, des noms de variables ou des messages d'exception, qui pourraient aider un attaquant à comprendre la structure ou la logique de l'application web.
- Enfin, "log_errors = On" active la journalisation des erreurs dans un fichier de log dédié. Cela permet de conserver un historique des incidents applicatifs sans exposer ces informations aux utilisateurs finaux, facilitant ainsi le diagnostic et la correction des erreurs par les administrateurs, tout en maintenant un haut niveau de discrétion.

L'ensemble de ces paramètres a été appliqué progressivement après chaque redémarrage du service PHP-FPM ("systemctl restart php7.4-fpm") et de celui d'Apache2 ("systemctl restart apache2"), afin de s'assurer que les fonctionnalités critiques n'étaient pas affectées. Ces ajustements contribuent à renforcer la résilience globale du serveur face aux attaques exploitant les failles applicatives ou les erreurs de configuration courantes.

Le service NFS permet le partage de fichiers à travers le réseau, mais il expose également la machine à plusieurs vecteurs d'attaque s'il est mal configuré, notamment en ce qui concerne les droits d'accès, l'authentification, et les permissions des clients. La configuration du partage NFS sur le serveur SE4FS a été définie dans le fichier "/etc/exports" comme suit:

```
/home (rw,sync,no_subtree_check,no_root_squash,sec=sys,sec=krb5)
/var/sambaedu (rw,sync,no_subtree_check,no_root_squash,sec=sys,sec=krb5)
```

Chaque ligne représente un répertoire exporté, suivi des options de contrôle d'accès entre parenthèses. Voici le détail des options appliquées:

- "rw": Autorise la lecture et l'écriture par les clients NFS.
- "sync": Oblige le serveur à écrire les modifications sur le disque avant d'envoyer une réponse au client, garantissant l'intégrité des données.
- "no_subtree_check": Désactive la vérification que les fichiers accédés par un client appartiennent bien à l'arborescence exportée. Cela évite certains problèmes liés aux mouvements de fichiers, tout en étant un peu moins strict.
- "no_root_squash": Cette option permet au client NFS d'agir avec les privilèges root sur les fichiers du serveur. C'est une configuration potentiellement dangereuse en environnement non maîtrisé. Elle est acceptable uniquement si les clients sont de confiance et que le réseau est sécurisé.
- "sec=sys": Utilise le système d'authentification classique basé sur les UID/GID locaux. C'est le mode par défaut, mais il ne fournit aucune authentification forte.
- "sec=krb5": Active l'authentification via Kerberos, permettant de renforcer la sécurité des communications entre le client et le serveur, en s'assurant que seuls les utilisateurs authentifiés via le domaine Kerberos puissent accéder aux ressources partagées.

Ces options combinées permettent un fonctionnement souple pour les postes du domaine tout en ajoutant une couche de sécurité via Kerberos. Toutefois, la présence de "no_root_squash" impose une vigilance particulière, car ce paramètre devrait idéalement être supprimé ou remplacé par "root_squash" dans un contexte plus exposé ou hétérogène. Une fois les modifications du fichier "/etc/exports réalisées", la commande suivante a été exécutée pour recharger la configuration sans redémarrer le service NFS:

```
exportfs -ra
```

Au-delà du partage de fichiers via NFS, le serveur SE4FS remplit plusieurs rôles essentiels au sein de l'infrastructure. Contrairement à SE4AD, davantage centré sur l'authentification et la gestion des

comptes utilisateurs, SE4FS est dédié aux services de fichiers, d'impression, de communication réseau, et de supervision, ce qui se reflète clairement dans sa configuration réseau et firewall.

Parmi les ports autorisés, on retrouve à nouveau les ports 137 et 138 en UDP, ainsi que les ports 139 et 445 en TCP, tous indispensables au bon fonctionnement de Samba. Samba implémente le protocole SMB pour le partage de fichiers et d'imprimantes. Les ports 137 et 138 sont utilisés pour le service NetBIOS Name Service et Datagram Service, servant à la résolution de noms NetBIOS et à la distribution de messages dans le réseau local. Les ports 139 et 445 permettent les échanges directs de fichiers, les authentifications utilisateur et la gestion des autorisations d'accès. Le port 6602, tout comme SE4AD en TCP est également ouvert pour le service SSH.

Contrairement au SE4AD, le serveur SE4FS autorise les flux web via les ports 80 et 443 en TCP. Le port 80 sert aux connexions HTTP non chiffrées, tandis que le port 443 est utilisé pour HTTPS, qui assure une communication sécurisée par chiffrement SSL/TLS. Cela indique que le serveur héberge une ou plusieurs interfaces web, par exemple pour la gestion des utilisateurs, le suivi d'activités, ou des tableaux de bord.

Le port 67 en UDP est utilisé pour le service DHCP, qui permet des paramètres de configuration réseau tels que la passerelle ou les serveurs DNS. Le port 69 en UDP est utilisé par le service TFTP, souvent mis en œuvre pour le boot PXE des machines sans système d'exploitation, ou pour transférer de petits fichiers de configuration sans authentification.

FreeRADIUS est aussi installé sur SE4FS. Ce serveur d'authentification réseau utilise les ports 1812 en UDP pour les requêtes d'authentification et 1813 pour la comptabilisation, qui permet de tracer les connexions et les durées d'utilisation. FreeRADIUS peut interagir avec LDAP, Kerberos ou une base SQL pour vérifier les identifiants des utilisateurs.

La base MariaDB est également présente sur ce serveur, écoutant sur le port 3306 TCP, afin de stocker les données nécessaires à plusieurs services: logs RADIUS, comptes utilisateurs, configurations internes ou même pages web dynamiques.

Enfin, le serveur prend en charge l'impression réseau via CUPS (Common UNIX Printing System), un service très utilisé dans les environnements Linux pour la gestion centralisée des files d'attente d'imprimantes. Il utilise le port 631 en TCP et UDP pour permettre aux postes clients d'envoyer des travaux d'impression, de consulter l'état des imprimantes ou de modifier leur configuration.

3.1.4 Problèmes rencontrés et leur solution

L'un des incidents majeurs rencontrés au cours de la mise en place de l'infrastructure a concerné un dysfonctionnement de l'accès aux comptes utilisateurs Samba depuis les postes élèves; concrètement, les utilisateurs ne parvenaient plus à se connecter à leur session Windows. Ce problème est apparu après une phase de configuration du pare-feu UFW sur SE4AD et SE4FS. La première étape a consisté à vérifier si le pare-feu pouvait être à l'origine de la panne. Pour cela, j'ai désactivé temporairement UFW sur les deux serveurs à l'aide de la commande:

```
sudo ufw disable
```

Immédiatement après cette action, l'accès aux comptes utilisateurs depuis les postes clients a de nouveau fonctionné correctement, ce qui m'a permis de confirmer que le problème était bien lié aux règles UFW mises en place sur les serveurs. Afin d'identifier lequel des deux serveurs était réellement en cause, j'ai procédé par élimination:

- J'ai d'abord laissé UFW activé sur SE4AD et désactivé sur SE4FS, puis effectué un test de connexion, qui a résulté en un échec.
- Ensuite, j'ai inversé la configuration en activant UFW sur SE4FS et en le désactivant sur SE4AD, ce qui a permis une connexion réussie.

Cette série de tests a clairement montré que le pare-feu du serveur SE4AD bloquait un ou plusieurs ports nécessaires au bon fonctionnement de l'authentification et de la connexion des clients au domaine.

Pour investiguer davantage, j'ai activé la journalisation UFW sur SE4AD à l'aide de la commande :

```
sudo ufw logging on
```

Puis, j'ai utilisé la commande suivante pour consulter les logs en temps réel (Figure 6) :

```
sudo tail -f /var/log/ufw.log
```

Là, j'ai pu observer plusieurs lignes bloquées par le pare-feu. Deux d'entre elles ont été particulièrement révélatrices :

- Une tentative de connexion TCP sur le port 53, bloquée :
[UFW BLOCK] IN=ens18 OUT= MAC=... SRC=126.10.3.62 DST=126.10.3.201 ...
PROTO=TCP SPT=55761 DPT=53 ...

Cette ligne montrait qu'un client essayait d'établir une connexion TCP vers le port 53 (DNS) du serveur SE4AD. Or j'avais autorisé uniquement le trafic UDP sur le port 53, considérant que la majorité des requêtes DNS utilisent UDP. Ce n'était pas une erreur en soi, mais un oubli courant : en réalité, les clients peuvent passer en TCP pour certaines réponses DNS trop volumineuses ou lors de transferts de zones.

- Une autre ligne montrait un blocage en UDP sur le port 389 :
[UFW BLOCK] IN=ens18 OUT= MAC=... SRC=126.10.3.62 DST=126.10.3.201 ...
PROTO=UDP SPT=60322 DPT=389 ...

Le port 389 est utilisé par LDAP, essentiel à l'annuaire Active Directory. Là encore, j'avais autorisé le port 389 en TCP dans mes règles UFW, ce qui couvre la plupart des échanges LDAP classiques (authentification, requêtes, gestion d'objets de l'annuaire, etc...). Cependant, Windows utilise aussi UDP sur le port 389 pour la découverte des contrôleurs de domaine, notamment via le protocole CLDAP (Connection-less LDAP).

```
Apr 10 10:17:32 se4ad kernel: [85793.085872] [UFW BLOCK] IN=ens18 OUT= MAC=ba:6a:6d:8e:ab:7f:8a:e1:b4:18:ed:70:08:00 SRC=126.10.3.200 DST=126.10.3.201 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=43125 DF PROTO=TCP SPT=53794 DPT=22 WINDOW=64240 RES=0x00 SYN URG=0
Apr 10 10:17:55 se4ad kernel: [85816.587992] [UFW BLOCK] IN=ens18 OUT= MAC=ba:6a:6d:8e:ab:7f:c4:65:16:14:f9:79:08:00 SRC=126.10.3.74 DST=126.10.3.201 LEN=275 TOS=0x00 PREC=0x00 TTL=128 ID=53732 PROTO=UDP SPT=50252 DPT=389 LEN=255
Apr 10 10:18:10 se4ad kernel: [85831.773721] [UFW BLOCK] IN=ens18 OUT= MAC=ba:6a:6d:8e:ab:7f:c4:65:16:14:f9:22:08:00 SRC=126.10.3.62 DST=126.10.3.201 LEN=52 TOS=0x00 PREC=0x00 TTL=128 ID=47669 DF PROTO=TCP SPT=55761 DPT=53 WINDOW=65535 RES=0x00 SYN URG=0
Apr 10 10:18:30 se4ad kernel: [85851.222518] [UFW BLOCK] IN=ens18 OUT= MAC=ba:6a:6d:8e:ab:7f:c4:65:16:14:f9:22:08:00 SRC=126.10.3.62 DST=126.10.3.201 LEN=275 TOS=0x00 PREC=0x00 TTL=128 ID=47675 PROTO=UDP SPT=60322 DPT=389 LEN=255
```

Figure 6: Résultat partiel de la commande `sudo tail -f /var/log/ufw.log` avec les deux erreurs importantes

Ces deux logs m'ont permis de cibler précisément les ports et protocoles manquants dans ma configuration UFW. Pour corriger la situation, j'ai donc ajouté les règles suivantes :

```
sudo ufw allow from 126.0.0.0/8 to any port 53 proto tcp
```

```
sudo ufw allow from 126.0.0.0/8 to any port 389 proto udp
```

Avec ces deux commandes, j'ai autorisé le trafic DNS TCP depuis le réseau local, et le trafic LDAP UDP, également en provenance du même réseau. Après avoir fait un essai depuis un compte élève test, les connexions Samba fonctionnaient de nouveau normalement depuis tous les postes.

Lors de la tentative de mise à jour du serveur SE4AD avec la commande "`apt update -y && apt full-upgrade -y`", une erreur est apparue liée à un dépôt invalide dans le fichier de sources APT.

Après analyse du fichier `"/etc/apt/sources.list"` via nano, j'ai identifié une ligne problématique à la ligne 3. J'ai donc décidé de commenter cette ligne (en ajoutant un `#` en début) afin d'éviter toute tentative de contact avec ce dépôt lors des prochaines mises à jour.

La même vérification a été effectuée sur SE4FS. En ouvrant également le fichier `"/etc/apt/sources.list"`, j'ai constaté la présence de la même ligne défectueuse. Elle a été commentée de la même manière avant de relancer les mises à jour. Cependant, une autre problématique est survenue avec la connexion SSH. Le port par défaut avait été changé sur SE4AD, remplacé par le port 6602. Cela empêchait les connexions SSH classiques, nécessaires pour certaines mise-à-jour. Pour contourner cela sans devoir à chaque fois spécifier manuellement le port, j'ai créé un fichier de configuration SSH personnel à l'utilisateur root (`"nano ~/.ssh/config"`) dans lequel j'ai défini les paramètres suivants:

```
Host se4ad
HostName 126.10.3.201
Port 6602
User root
```

Grâce à cette configuration, je pouvais me connecter directement avec `"ssh se4ad"` sans me soucier du port non standard. Un autre problème est cependant apparu: une alerte concernant l'empreinte de la clef SSH du serveur, qui n'était pas reconnue. Pour résoudre cela, j'ai copié manuellement l'entrée correspondant à la clef de SE4AD depuis le fichier `"~/.ssh/known_hosts"` (fichier local d'un utilisateur) vers le fichier global du système `"/etc/ssh/ssh_known_hosts"`, sous la forme suivante:

```
126.10.3.201 ssh-rsa <clé>
```

Cela fonctionnait, mais pas durablement: le fichier `"/etc/ssh/ssh_known_hosts"` se vidait régulièrement, probablement à la suite de commandes d'administration ou de scripts automatisés. Pour pallier cela de manière temporaire, j'ai mis en place une tâche cron (via la commande `"crontab -e"`) qui vérifiait toutes les minutes si le fichier était vide, et le recomplétait automatiquement:

- ```
* * * * * [! -s /etc/ssh/ssh_known_hosts] && echo "126.10.3.201 ssh-rsa <clé>" >>
/etc/ssh/ssh_known_hosts
```
- `* * * * *`: exécute la tâche chaque minute.
  - `[ ! -s /etc/ssh/ssh_known_hosts ]`: vérifie si le fichier est vide.
  - `&& echo "... " >> ...`: ajoute la clef dans le fichier si c'est le cas.

Cette solution m'a permis de stabiliser temporairement la connexion SSH sans devoir accepter manuellement la clef à chaque tentative. Toutefois, cette approche n'était pas idéale à long terme. En effet, après une mise à jour ultérieure, le problème est réapparu sous une autre forme, ce qui m'a conduit à envisager une solution plus robuste et définitive pour la gestion des clefs SSH dans l'environnement.

Afin de faciliter l'administration, j'ai mis en place plusieurs commandes personnalisées intégrées à mon environnement shell. Parmi elles:

- `"fixssh"`: permet de modifier dynamiquement le port d'écoute du service SSH dans le fichier `"/etc/ssh/sshd_config"`, de relancer le service, et d'actualiser le pare-feu si besoin. C'est particulièrement utile pour basculer temporairement vers un port plus standard (comme le 22) lors d'opérations sensibles, comme les mises à jour système (voir l'annexe p.7 pour le script complet).
- `"servicerunning"`: affiche les services en cours d'exécution via `"systemctl list-units --type=service --state=running"`, pour un aperçu rapide de l'état du système.
- `"listen"`: liste tous les ports actuellement en écoute à l'aide de `"ss -tln"`, permettant de vérifier la surface d'exposition réseau du serveur.

J'ai également modifié certaines commandes natives pour améliorer leur lisibilité en y ajoutant des couleurs (`ls`, `ll`, `l`, `grep`, `ip`), ce qui permet de repérer plus facilement les fichiers, répertoires, ou correspondances dans les retours de commandes. Toutes ces modifications ont été appliquées via la commande `"source ~/.bashrc"` après édition.

Pour accompagner ces commandes, j'ai voulu créer une vraie page de manuel personnalisée, accessible par "man <nom\_commande>". Pour cela, j'ai d'abord créé le fichier de documentation:

```
mkdir -p ~/.local/share/man/man1
touch ~/.local/share/man/man1/<bom_commande>.1
```

Après avoir écrit le contenu dans un format respectant la structure classique des pages man, j'ai généré la base de données man locale:

```
mandb ~/.local/share/man/
```

Cependant, l'appel à "man fixssh" ne fonctionnait pas initialement. Après analyse, le problème venait du fait que le MANPATH ne pointait pas vers mon répertoire local. Pour corriger cela, j'ai ajouté cette ligne dans mon ".bashrc":

```
export MANPATH="$HOME/.local/share/man:$MANPATH"
```

Une fois cela fait, la page était bien reconnue et affichée comme n'importe quelle autre documentation système.

Un autre problème récurrent est apparu lors des mise-à-jour de SE4FS: la connexion SSH depuis SE4FS vers SE4AD échouait si le port utilisé n'était pas temporairement rouvert. En effet, comme SE4AD n'écoute pas sur le port SSH par défaut, mais sur un port personnalisé, certaines opérations comme "apt upgrade" échouaient car elles nécessitent d'établir une connexion vers SE4AD.

La mise en place d'un script de mise à jour sécurisée m'a donc semblé nécessaire. Ce script permet de temporairement basculer SE4AD vers le port 22, le temps de la mise à jour, puis de restaurer la configuration d'origine, tout en créant des copies des fichiers de sauvegardes en cas d'erreur lors de leur modification. Cette automatisation permet d'accélérer les procédures de maintenance, et d'optimiser la gestion des règles UFW en assurant un retour à l'état sécurisé initial une fois la tâche effectuée. Pour rendre cela possible, j'ai généré une paire de clés SSH sur SE4FS avec:

```
ssh-keygen -t rsa -b 4096
```

Puis j'ai autorisé la connexion sans mot de passe vers SE4AD via:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub -p 6602 root@126.10.3.201
```

Ainsi, SE4FS peut désormais établir automatiquement une connexion SSH vers SE4AD, même dans le cadre d'un script, sans nécessiter d'intervention humaine. Voici comment le script fonctionne (voir l'annexe p.8 pour le script complet):

- Il commence par une connexion SSH au serveur SE4AD, en utilisant le port non standard 6602. Une fois connecté via ce port sécurisé, la première étape consiste à autoriser temporairement le port 22 dans le pare-feu UFW à l'aide de la commande "ufw allow 22". Cela permet de rétablir un accès SSH classique, mais qui reste temporaire.
- Une fois la règle ajoutée, le pare-feu est rechargé via "ufw reload" afin que la nouvelle configuration prenne effet immédiatement. L'objectif est d'ouvrir brièvement le port 22 le temps de la mise à jour. Ensuite, on exécute la commande personnalisée "fixssh -p 22", qui modifie la configuration du service SSH pour qu'il écoute à nouveau sur le port 22 au lieu de 6602. Cette commande modifie le fichier de configuration "/etc/ssh/sshd\_config" pour y changer la directive Port, suivie d'un redémarrage du service SSH avec "systemctl restart ssh" pour appliquer le changement.
- Une petite pause de quelques secondes ("sleep 5") est insérée dans le script pour laisser le temps au démon SSH de redémarrer proprement sur le nouveau port.
- Une fois cette transition effectuée, on lance la mise à jour du système SE4FS avec les commandes "apt update && apt upgrade -y". Cette mise à jour nécessite que SE4AD soit joignable en SSH tout au long de l'opération, car certaines tâches automatisées ou scripts d'administration peuvent s'appuyer sur une communication directe entre les deux serveurs.

- Une fois la mise à jour de SE4FS terminée, une nouvelle connexion SSH est établie vers SE4AD, mais cette fois via le port 22, désormais actif. Le script réautorise alors le port 6602 avec la commande "ufw allow 6602", afin de préparer le retour à la configuration initiale. Il relance ensuite "fixssh -p 6602", qui remet le service SSH à écouter sur le port 6602.
- Pour finaliser proprement l'opération, la règle temporaire qui autorisait le port 22 est supprimée avec la commande "ufw delete allow 22". On recharge une nouvelle fois le pare-feu avec "ufw reload" pour appliquer les modifications, puis on redémarre le service SSH avec "systemctl restart ssh" afin d'appliquer le retour sur le port personnalisé.

Pour aller plus loin, j'ai mis en place une intégration directe de ce processus dans l'environnement de travail de SE4FS. J'ai modifié le fichier ".bashrc" (voir l'annexe p.8 pour le .bashrc complet) pour y ajouter une commande personnalisée, ou plutôt un argument de commande personnalisé "apt upgrade -SSH". Lorsqu'elle est exécutée, elle déclenche automatiquement le script "./secure\_upgrade.sh", qui applique toute la procédure de mise à jour sécurisée, gère les basculements de port SSH et les règles UFW, puis restaure l'état initial une fois l'opération terminée. Cette approche permet d'éviter les erreurs classiques liées aux restrictions de port durant une mise à jour, tout en rendant le processus aussi simple qu'une commande unique à exécuter.

## 3.2 Gestion centralisée des postes et des utilisateurs

### 3.2.1 Contexte

L'une des problématiques récurrentes relevées par l'équipe technique concernait la gestion et l'administration des postes clients ainsi que des comptes utilisateurs. Le parc informatique du campus est composé d'environ une centaine d'ordinateurs portables élèves, 110 portables pour le personnel, une soixantaine de postes fixes, et près d'une centaine de tablettes. Ces équipements sont répartis entre différents bâtiments, salles de cours, laboratoires et bureaux administratifs. Chaque machine peut être utilisée par plusieurs profils selon le rôle de l'utilisateur, qu'il s'agisse d'élèves, d'enseignants ou de membres du personnel. Les besoins d'accès sont variés: certains doivent accéder à des dossiers partagés, d'autres à des imprimantes réseau, à des logiciels pédagogiques spécifiques, ou encore à des services internes comme l'interface SambaEdu.

Avant la mise en place d'un système centralisé, la gestion des machines et des comptes se faisait manuellement ou de manière partiellement automatisée, ce qui consommait beaucoup de temps et multipliait les risques d'erreurs. La mise en oeuvre uniforme des règles de sécurité, le suivi des droits ou encore la traçabilité des connexions étaient difficiles, notamment avec le renouvellement constant des utilisateurs, entre les nouveaux élèves chaque année, les enseignants temporaires et le personnel en mouvement.

Pour répondre à ces défis, le campus a mis en place une architecture centralisée fondée sur SambaEdu4, une solution libre dérivée de Samba 4 qui permet d'implémenter un contrôleur de domaine Active Directory sous GNU/Linux. Cette solution repose sur un serveur principal, SE4AD, tournant sous Debian, qui centralise la gestion des comptes utilisateurs, des groupes, des ordinateurs ainsi que des politiques de sécurité grâce à des GPO. Chaque poste client est intégré au domaine et peut donc être administré à distance. Les utilisateurs, quel que soit leur rôle, peuvent se connecter à n'importe quel poste du réseau en retrouvant automatiquement leur environnement de travail: dossiers réseau mappés, ressources partagées, scripts d'ouverture de session exécutés automatiquement et paramètres cohérents.

À mon arrivée, cette infrastructure était déjà en place, mais certaines pratiques restaient manuelles ou peu formalisées. J'ai commencé par auditer le système existant: j'ai étudié la structure de l'annuaire LDAP, la logique des unités d'organisation (OU) utilisées pour classer les machines et les utilisateurs, et les stratégies appliquées via les GPO. À partir de cet audit, j'ai participé à une réorganisation de la structure: création de nouvelles OU pour mieux refléter les groupes réels (par

niveau scolaire, par fonction, etc...), amélioration de la logique des groupes utilisateurs, mise en place de règles plus strictes pour les connexions. Un des points les plus sensibles concernait la gestion des profils utilisateurs: j'ai aidé à déterminer pour quels groupes il était préférable d'utiliser des profils itinérants (enseignants, personnel) et pour lesquels il valait mieux conserver des profils locaux ou temporaires (élèves), dans le but de garantir de bonnes performances tout en respectant les besoins de chacun.

L'administration centralisée nécessitait aussi une gestion rigoureuse des données et des sauvegardes. Pour cela, un NAS Synology a été intégré à l'infrastructure. Il remplit plusieurs rôles: héberger les dossiers partagés accessibles via le réseau, stocker les profils utilisateurs itinérants, et servir de point central pour les sauvegardes. Les sauvegardes sont organisées à plusieurs niveaux: celles des données utilisateur, celles des configurations du contrôleur de domaine, et celles des ressources critiques du réseau. Des snapshots réguliers sont effectués à l'aide d'outils tels qu'Active Backup for Business, permettant de restaurer rapidement une machine ou un serveur complet en cas de problème. Cette stratégie assure à la fois une meilleure résilience et une traçabilité renforcée.

### **3.2.2 Plus de détail sur les outils et technologies utilisés**

Durant mon stage, j'ai été amené à travailler sur la mise en place et l'administration des stratégies de groupe dans un environnement hybride, combinant un serveur Debian fonctionnant comme un contrôleur de domaine Samba (équivalent à un Active Directory Windows) et des postes clients sous Windows. La gestion centralisée via GPO était essentielle dans cet environnement scolaire, afin d'assurer une homogénéité des configurations, une sécurité renforcée, et une facilité de maintenance, notamment pour des élèves et enseignants changeant régulièrement de salle ou de machine. Pour cela, j'ai utilisé les outils RSAT (Remote Server Administration Tools) depuis un poste Windows, ce qui m'a permis d'accéder à deux consoles majeures: "Utilisateurs et ordinateurs Active Directory" (ADUC) pour organiser les comptes et machines dans des unités d'organisation (OU), et la "Console de gestion des stratégies de groupe" (GPMC) pour créer, appliquer et tester des stratégies ciblées.

En parallèle de la gestion des politiques de groupe, une attention particulière a été portée à la centralisation des sauvegardes. Un NAS Synology a été intégré à l'infrastructure pour servir de serveur de fichiers dédié aux sauvegardes régulières des serveurs et des postes critiques. Ce système de centralisation a permis d'automatiser les sauvegardes grâce à une configuration via interface, d'assurer une meilleure résilience des données, et de limiter les risques de perte en cas de défaillance. Ce système, combiné à la gestion des GPO, a renforcé l'efficacité du réseau et a permis de standardiser le poste de travail tout en réduisant les interventions techniques manuelles.

### **3.2.3 Tâches effectués**

J'ai donc été chargé de concevoir, déployer et tester une série de stratégies de groupe visant à renforcer la sécurité et la stabilité des postes clients, principalement utilisés par des élèves. Une des premières actions a consisté à restreindre l'accès aux éléments critiques de Windows, comme le Panneau de configuration et les Paramètres système. Ces interfaces permettent en effet d'accéder à des fonctions sensibles (réseau, gestion des utilisateurs, désinstallation de logiciels, etc...) qu'il était impératif de bloquer pour éviter toute manipulation non autorisée. En activant les politiques correspondantes dans la configuration utilisateur via la console GPMC, j'ai pu interdire l'accès à ces menus tout en garantissant une expérience utilisateur fluide. Les élèves, lorsqu'ils tentaient d'accéder à ces fonctions, voyaient désormais un message indiquant que l'accès était bloqué par l'administrateur du système.

Une autre priorité essentielle a été le blocage des consoles de commande comme CMD et PowerShell. Ces outils sont puissants et permettent de lancer des commandes systèmes, d'accéder au registre, de manipuler les fichiers ou de contourner certaines restrictions si leur usage n'est pas

contrôlé. J'ai donc activé plusieurs politiques dans les modèles d'administration: pour CMD, j'ai interdit son ouverture et l'exécution des scripts .bat ; pour PowerShell, j'ai combiné l'interdiction d'exécuter des scripts .ps1 avec une stratégie logicielle interdisant purement l'exécution des binaires powershell.exe et pwsh.exe. Cela a permis de créer une double barrière, empêchant aussi bien les utilisateurs que les logiciels malveillants d'exploiter cette interface. Toujours dans cette logique, j'ai bloqué l'accès à l'Éditeur du Registre (regedit), dont l'usage peut compromettre l'intégrité du système. Toutes ces restrictions ont été testées sur des postes clients via des comptes élèves, et affinées en fonction des retours terrain, avec forçage des GPO via la commande gpupdate /force.

En parallèle de ces politiques, j'ai mis en place un système complet de sauvegarde automatisée, afin d'assurer la résilience de l'infrastructure et la protection des données critiques. Pour cela, j'ai déployé une instance de Proxmox, une solution de virtualisation open source, sur un serveur dédié. Cette plateforme héberge notamment les machines virtuelles SambaEdu et d'autres services clés de l'infrastructure réseau. Pour stocker les sauvegardes, j'ai utilisé un NAS Synology, configuré pour être accessible via NFS. La liaison entre Proxmox et le NAS s'est faite en montant le partage NFS comme datastore de sauvegarde (Figure 7).

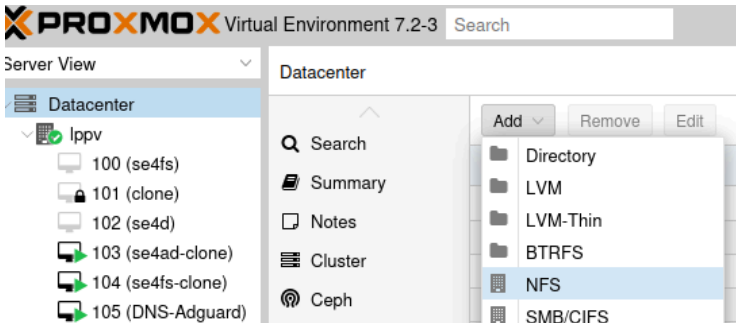


Figure 7: Ajout du Synology via NFS depuis Proxmox

J'ai programmé dans l'interface de Proxmox des sauvegardes complètes des VM tous les trois jours, avec une rétention glissante pour ne conserver que les versions utiles (Figure 8). Cela permet de garantir une reprise rapide d'activité en cas de corruption, de mauvaise manipulation ou de panne. Le choix du protocole NFS a été motivé par sa simplicité de mise en œuvre dans un environnement Linux, sa compatibilité avec Proxmox, et sa capacité à gérer les transferts de gros volumes de données en réseau local avec de bonnes performances.

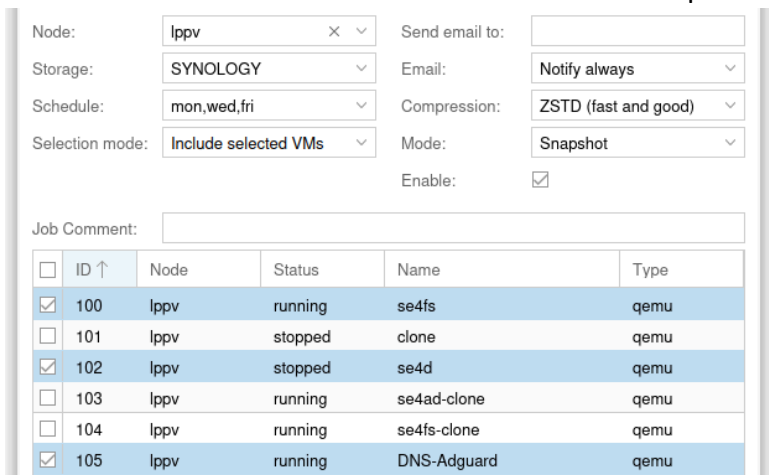


Figure 8: Programmation de la sauvegarde sur Proxmox

Du côté du NAS, j'ai aussi activé des snapshots internes et vérifié que les sauvegardes étaient bien accessibles depuis une machine d'administration externe, pour permettre un contrôle rapide de leur bon déroulement. Grâce à cette solution, l'ensemble des services critiques (contrôleur de

domaine, services web internes, pare-feu, etc...) bénéficient désormais d'un plan de secours automatisé, sans intervention manuelle.

### 3.2.4 Problèmes rencontrés et leur solution

Au cours du déploiement des stratégies de groupe et de la gestion du domaine, plusieurs problèmes techniques ont été rencontrés. L'un des premiers défis a été lié à des incompatibilités entre les GPO configurées et les versions de Windows installées sur les postes clients. Les machines du parc étant équipées de Windows 11 Pro, certaines stratégies conçues pour les éditions Entreprise ou Education ne s'appliquaient pas correctement, voire étaient totalement ignorées. Cela concernait par exemple certaines restrictions d'accès aux paramètres avancés, ou encore des configurations système spécifiques. Cette incompatibilité a exigé une vérification minutieuse de chaque stratégie appliquée, afin de s'assurer qu'elle avait bien un effet sur les machines concernées. Dans les cas où une GPO ne produisait aucun effet, elle a été modifiée ou remplacée par une alternative compatible avec l'édition Pro.

Un autre problème majeur est apparu suite à une migration du serveur Samba vers un nouveau domaine: plusieurs postes clients perdaient leur canal de communication sécurisé avec le contrôleur de domaine. Bien qu'ils soient encore visibles dans l'interface Active Directory, ils étaient incapables de s'y authentifier correctement, ce qui empêchait l'application des GPO et provoquait des erreurs à la connexion utilisateur. Plutôt que de réintégrer chaque poste manuellement, une solution automatisée a été mise en place sous la forme d'un script .bat contenant une commande PowerShell. Ce script, lancé localement après connexion en administrateur local, permettait de réparer le canal sécurisé en réinitialisant la relation de confiance avec le domaine. Il était exécuté depuis une clé USB insérée sur le poste, après identification de la lettre du lecteur. Dans les cas où le script échouait, des causes récurrentes ont été identifiées, notamment une mauvaise connexion réseau (câble Ethernet mal branché) ou un DNS incorrect. Lorsque nécessaire, l'adresse DNS du poste était modifiée manuellement pour pointer vers celle du serveur SambaEdu (126.10.3.201). Cette méthode a permis de restaurer rapidement la communication avec le domaine sur la majorité des machines concernées.

Pour renforcer le suivi et la supervision de l'infrastructure, afin de toujours savoir si un serveur perd connexion, j'ai également déployé des outils de monitoring sur les serveurs. Deux solutions principales ont été retenues: Glances et Netdata. Glances est un outil léger en ligne de commande qui fournit une vision synthétique en temps réel de l'état du système: charge CPU, mémoire, réseau, espace disque, services actifs, etc... Il a été installé sur plusieurs machines critiques, notamment sur le serveur SambaEdu et le serveur de sauvegarde. Cet outil m'a permis de détecter rapidement certaines anomalies, comme des surcharges temporaires ou des services inactifs.

En complément, j'ai mis en place Netdata, un outil plus visuel et interactif, accessible depuis un navigateur web. Netdata propose une interface graphique complète, avec des tableaux de bord dynamiques pour chaque ressource du système. Son installation sur les serveurs Debian a permis d'avoir un suivi en temps réel de l'activité, avec historique, alertes automatiques, et vues détaillées sur les processus, le trafic réseau, les entrées/sorties disques, ou encore l'état du système de fichiers. Grâce à lui, j'ai pu corrélérer des pics d'utilisation avec certaines opérations de sauvegarde sur Proxmox, et anticiper des problèmes liés à la saturation disque ou à l'activité réseau.

Un autre point d'optimisation important abordé concernait la fluidité et la stabilité des connexions des utilisateurs aux services hébergés sur les machines virtuelles. En effet, des lenteurs lors des connexions, notamment lors de l'application des GPO ou de l'ouverture de session, ont été constatées sur plusieurs postes clients, particulièrement lorsque plusieurs VMs partageaient la

même interface réseau physique. Ce type de congestion réseau est courant lorsque plusieurs machines virtuelles s'appuient simultanément sur un unique bridge réseau, ce qui peut saturer le réseau, et causer des délais de réponse accrus ou des déconnexions intermittentes.

Pour pallier ce problème, j'ai mis en place une configuration réseau dédiée sur le serveur Proxmox. L'objectif était de répartir la charge réseau en attribuant une interface Ethernet distincte à chaque VM critique, notamment les serveurs SambaEdu, le serveur de sauvegarde, et ceux liés à la supervision. Pour cela, j'ai modifié manuellement le fichier `"/etc/network/interfaces"` sur le serveur hôte, en déclarant plusieurs interfaces physiques, chacune associée à un bridge réseau propre (Figure 9). Une fois les nouvelles interfaces configurées et activées via `systemctl restart networking`, celles-ci étaient disponibles dans l'interface de gestion de Proxmox.

```
SE4FS
auto vmbr1
iface vmbr1 inet manual
 bridge-ports ens7f0
 bridge-stp off
 bridge-fd 0

SE4AD
auto vmbr2
iface vmbr2 inet manual
 bridge-ports ens7f1
 bridge-stp off
 bridge-fd 0

DNS
auto vmbr3
iface vmbr3 inet manual
 bridge-ports ens4f0
 bridge-stp off
 bridge-fd 0
```

Figure 9: Contenu de `/etc/network/interfaces`

L'étape suivante consistait à attribuer chaque bridge à une VM spécifique. Depuis l'interface Proxmox, en accédant aux paramètres d'une VM (onglet Hardware > Network Device), j'ai modifié le bridge associé à l'interface réseau de la machine virtuelle. Ainsi, chaque serveur tournant dans une VM disposait de sa propre liaison réseau dédiée, sans conflit ni saturation avec les autres (Figure 10). Ce découplage réseau a eu un effet immédiat sur la réactivité des services: les connexions aux comptes utilisateurs se faisaient plus rapidement, l'application des GPO était fluide, et les tâches de sauvegarde ou de supervision n'interféraient plus avec les services critiques.

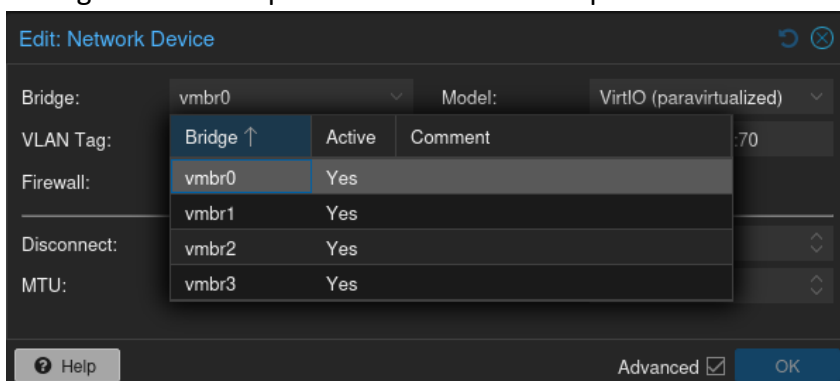


Figure 10: Assignment des interfaces réseaux aux VMs





## 4 Conclusion

Ce stage m'a offert l'opportunité de contribuer activement à l'amélioration d'une infrastructure informatique existante, en apportant des réponses concrètes à des enjeux de sécurité, de performance et d'organisation. La mise en place de stratégies de groupe ciblées, l'optimisation réseau via Proxmox, la gestion centralisée des sauvegardes sur un NAS Synology ou encore l'intégration d'outils de supervision comme Glances et Netdata m'ont permis de consolider mes compétences techniques tout en développant une démarche rigoureuse et structurée.

J'ai ainsi pu mettre en pratique de nombreuses notions abordées au sein de l'IUT Réseaux & Télécoms, notamment en administration système, en cybersécurité, en virtualisation ou encore en documentation technique. À travers la configuration de systèmes Debian, la gestion des services réseau, l'automatisation par scripts ou encore l'administration centralisée via Active Directory, ce stage a été l'occasion d'ancrer durablement mes acquis dans des situations réelles et complexes.

Au-delà des aspects purement techniques, cette expérience m'a permis de comprendre l'importance de la collaboration au sein d'une équipe informatique et d'anticiper les besoins évolutifs d'une infrastructure. Elle a également renforcé ma conviction sur la pertinence de la formation BUT Réseaux & Télécoms, qui allie solides bases théoriques, vision globale des systèmes d'information et préparation aux réalités du métier. Ce stage a ainsi constitué une passerelle essentielle entre le monde académique et le monde professionnel.



## 5 Remerciements

Je tiens à exprimer ma sincère reconnaissance à l'équipe du service informatique du Campus Provence Verte pour m'avoir accueilli avec bienveillance au sein de leur structure. Leur disponibilité, leur confiance et leurs conseils m'ont permis de progresser tout au long de ce stage, dans un environnement à la fois technique, stimulant et formateur.

Je remercie tout particulièrement Ioane Ducresson pour son accompagnement attentif, sa pédagogie et sa rigueur. Son encadrement m'a grandement facilité l'intégration et m'a permis de mettre concrètement en oeuvre les compétences acquises durant ma formation à l'IUT.

J'adresse également mes remerciements à l'ensemble de l'équipe pédagogique de l'IUT d'Aix-Marseille, et plus spécifiquement à Éric Würbel, pour la qualité des enseignements dispensés et leur lien direct avec les tâches effectuées au sein du stage, qui m'ont permis de l'aborder avec confiance.



## 6 Glossaire

**IT**, Technologie de l'information

**BUT**, Bachelor Universitaire de Technologie

**IUT**, Institut Universitaire de Technologie

**RAID 1**, Redundant Array of Independent Disks, niveau 1, Méthode de stockage qui duplique les données sur deux disques (miroir)

**software**, Logiciel ou ensemble de programmes exécutés par un ordinateur

**hardware**, Matériel ou composants physiques d'un système informatique

**IP**, Internet Protocol, Adresse identifiant un appareil sur un réseau

**SE4AD**, Serveur Éducatif 4 Active Directory, Serveur contrôleur de domaine basé sur Samba

**MariaDB**, Maria Database, Système de gestion de bases de données relationnelles libre

**SE4FS**, Serveur Éducatif 4 File Server, Serveur de fichiers dédié à la gestion réseau

**TFTP**, Trivial File Transfer Protocol, Protocole simple de transfert de fichiers sans authentification

**UNIX**, Uniplexed Information and Computing System, Système d'exploitation multitâche

**HTTP**, HyperText Transfer Protocol, Protocole de communication pour le web

**LXC**, Linux Containers, Technologie de virtualisation légère sous Linux

**UDP**, User Datagram Protocol, Protocole réseau rapide sans vérification d'erreurs

**TCP**, Transmission Control Protocol, Protocole réseau fiable avec vérification des données

**SMB**, Server Message Block, Protocole de partage de fichiers et d'imprimantes sous Windows

**démon**, Daemon, Processus système s'exécutant en arrière-plan sous Unix/Linux

**PHP**, Hypertext Preprocessor, Langage de script pour le développement web côté serveur

**NetBIOS**, Network Basic Input/Output System, Interface de communication réseau de bas niveau

**TLS**, Transport Layer Security, Protocole de sécurisation des échanges réseau

**LDAPS**, Lightweight Directory Access Protocol over SSL, Version sécurisée du protocole LDAP

**MySQL**, My Structured Query Language, Système de gestion de base de données open source

**PHP-FPM**, PHP FastCGI Process Manager, Gestionnaire PHP optimisé pour les serveurs web

**FastCGI**, Fast Common Gateway Interface, Protocole pour exécuter des scripts via un serveur web

**UID/GID**, User/Group Identifier, Identifiants numériques des utilisateurs et groupes sous Unix/Linux

**Boot PXE**, Preboot eXecution Environment, Démarrage d'un ordinateur via le réseau

**SQL**, Structured Query Language, Langage de requête pour les bases de données relationnelles

**CLDAP**, Connectionless LDAP, Variante de LDAP sur UDP pour la découverte de domaine

**GPO**, Group Policy Object, Ensemble de règles appliquées aux utilisateurs et ordinateurs dans un domaine Windows



## 7 Sitographie

**TuxCare.** (2023). What is Proxmox? <https://tuxcare.com/blog/what-is-proxmox/>

**DynFi.** (n.d.). Qu'est-ce que Proxmox ? <https://dynfi.com/proxmox/qu-est-ce-que-proxmox/>

**LinuxFr.** (2021). SambaEdu 4: une solution de serveurs pédagogiques libres basés sur GNU/Linux. <https://linuxfr.org/news/sambaedu-4-une-solution-de-serveurs-pedagogiques-libres-bases-sur-gnu-linux>

**SambaEdu.org.** (2023). Mise à jour vers Debian 12 Bookworm.

<https://www.sambaedu.org/Mise-a-jour-vers-Debian-12-Bookworm>

**YouTube.** (n.d.). Vidéo Proxmox & NAS Synology.

<https://www.youtube.com/watch?v=BLwRfhvBJ4&themeRefresh=1>

**Forum Proxmox.** (2024). Backup to NAS Synology.

<https://forum.proxmox.com/threads/backup-to-nas-synology.149630/>

**YouTube.** (n.d.). Installer et configurer Proxmox Backup Server.

<https://www.youtube.com/watch?app=desktop&v=hvnuRmroBaE>

**IT-Connect.** (2022). Corriger erreur de relation d'approbation.

<https://www.it-connect.fr/windows-comment-corriger-erreur-de-relation-approbation-voici-plusieurs-methodes/>

**Belginux.** (2023). Installer Proxmox Backup Server.

<https://belginux.com/installer-proxmox-backup-server/>

**Technonagib.** (2023). Ajouter disques SATA/USB à Proxmox VE.

<https://technonagib.fr/ajouter-disques-sata-usb-proxmox-ve/>

**IT-Connect.** (2021). Installer Proxmox VE 7.0 et créer sa première VM.

<https://www.it-connect.fr/comment-installer-proxmox-ve-7-0-et-creeer-sa-premiere-vm/>

**Dell Technologies.** (n.d.). Create a Virtual Disk on PowerEdge (System Setup).

<https://www.dell.com/support/kbdoc/en-ba/000178269/dell-poweredge-how-to-create-a-virtual-disk-through-the-system-setup-on-14g-servers>

**Microsoft.** (n.d.). RSAT - Remote Server Administration Tools.

<https://learn.microsoft.com/fr-fr/windows-server/remote/remote-server-administration-tools>

**Netdata.** (n.d.). What is Netdata? <https://learn.netdata.cloud/docs/overview/what-is-netdata>

**Glances.** (n.d.). Glances - An Eye on Your System. <https://nicolargo.github.io/glances/>

**MariaDB Foundation.** (n.d.). MariaDB - Open Source Database. <https://mariadb.org/>

**Synology.** (n.d.). Centre de connaissances Synology. <https://kb.synology.com/fr-fr>